A project report on

A CROSS-ATTENTION DEEP LEARNING FRAMEWORK FOR MULTIMODAL NEUROIMAGING FUSION IN VISION DEFECT DETECTION AND CLASSIFICATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

MISHTHI JAIN ([Reg. No.])

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

[November, 2026]

A CROSS-ATTENTION DEEP LEARNING FRAMEWORK FOR MULTIMODAL NEUROIMAGING FUSION IN VISION DEFECT DETECTION AND CLASSIFICATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

MISHTHI JAIN

Registration No.: [Reg. No.]

Course: BCSE497J

Guide: [Guide name, Designation]



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Vellore Institute of Technology, Chennai

Submission date: [November, 2026]

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "A Cross-Attention Deep Learning Framework for Multimodal Neuroimaging Fusion in Vision Defect Detection and Classification" submitted by MISHTHI JAIN ([Reg. No.]), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of [Guide name].

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: [dd Month yyyy] Signature of the Candidate

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "A Cross-Attention Deep Learning Framework for Multimodal Neuroimaging Fusion in Vision Defect Detection and Classification" is prepared and submitted by MISHTHI JAIN ([Reg. No.]) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: [Dr./Prof. Guide name]

Date:

Signature of the Examiner	Signature of the Examiner
Name:	Name:
Date:	Date:

Approved by the Head of Department,
Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Name: Dr. Tulasi Prasad Sariki

Date:

[Seal of the School]

ABSTRACT

Vision depends on an intact pathway from the retina to the visual cortex. Disorders such as glaucoma, optic neuritis and cortical visual impairment alter both the structure of this pathway, which is visible on magnetic resonance imaging (MRI), and the timing of its electrical response, which is recorded as the visual evoked potential (VEP). Most computational tools analyse only one of these sources of evidence. This project presents a research prototype, the Cross-Attention Fusion Network (CAFN), that combines a three-dimensional MRI volume with a one-dimensional VEP waveform to classify a case as Healthy Control, Glaucoma, or Optic Neuritis / Cortical Visual Impairment. A 3D residual network and a 1D convolutional network map the two inputs to 64 spatial and 16 temporal tokens in a shared 128-dimensional space. Multi-head cross-attention, in which spatial tokens query temporal tokens, fuses them, and a multi-task loss with two auxiliary unimodal heads supervises training. VEP conditioning uses zero-phase filtering so that P100 latency is preserved, and 3D Grad-CAM and an attention heatmap provide explanations. Because no public dataset offers the required diagnosis labels, the system was developed and verified on class-conditioned synthetic data. On 150 synthetic recordings, conditioning kept the P100 latency within one sample (4 ms) of the ground truth in 149 cases and attenuated the injected 50 Hz interference by 22 dB on average. The model has 1.06 million parameters, and an 86-test suite reaches about 95% statement coverage. No diagnostic accuracy is claimed. The work provides a tested foundation for evaluation on a labelled clinical cohort.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to [Guide name], [Designation], School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for [his/her] constant guidance, continual encouragement, understanding; more than all, [he/she] taught me patience in my endeavor. My association with [him/her] is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of [area of expertise].

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. Tulasi Prasad Sariki, Head of the Department, B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning) and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: [dd Month yyyy] Mishthi Jain

CONTENTS

ABSTRACT ..... **i**

ACKNOWLEDGEMENT ..... **ii**

CONTENTS ..... **iii**

LIST OF TABLES ..... **vi**

LIST OF FIGURES ..... **viii**

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE ..... **x**

CHAPTER 1 INTRODUCTION **1**

1.1 BACKGROUND 1

1.2 MOTIVATION AND CHALLENGES 2

1.2.1 UNIMODAL ISOLATION 2

1.2.2 CROSS-MODAL HETEROGENEITY 2

1.2.3 LIMITED INTERPRETABILITY 2

1.3 PROBLEM STATEMENT 3

1.4 OBJECTIVES AND RESEARCH QUESTIONS 3

1.4.1 OBJECTIVES 3

1.4.2 RESEARCH QUESTIONS 4

1.4.3 WORKING HYPOTHESES 4

1.5 SCOPE OF THE PROJECT 5

1.6 LIMITATIONS OF THE STUDY 5

1.7 CONTRIBUTIONS AND DISTINCTION FROM EXISTING WORK 5

1.8 ORGANISATION OF THE REPORT 6

CHAPTER 2 LITERATURE REVIEW **7**

2.1 INTRODUCTION 7

2.2 CLINICAL BACKGROUND 7

2.2.1 GLAUCOMA, OPTIC NEURITIS AND CORTICAL VISUAL IMPAIRMENT 7

2.2.2 THE VISUAL EVOKED POTENTIAL AND THE P100 COMPONENT 8

2.3 DEEP LEARNING FOR VOLUMETRIC NEUROIMAGING 8

2.4 DEEP LEARNING FOR ONE-DIMENSIONAL ELECTROPHYSIOLOGICAL SIGNALS 9

2.5 MULTIMODAL DATA FUSION 9

2.6 ATTENTION AND CROSS-ATTENTION 10

2.7 EXPLAINABILITY 11

2.8 PUBLIC DATA RESOURCES AND THE LABELLING GAP 11

2.9 RESEARCH GAP AND POSITION OF THIS PROJECT 12

CHAPTER 3 METHODOLOGY **13**

3.1 OVERVIEW OF THE APPROACH 13

3.2 DESIGN PRINCIPLES 14

3.3 TOOLS AND ENVIRONMENT 15

3.4 DATA SOURCES 15

3.5 SYNTHETIC DATA GENERATION 16

3.5.1 SYNTHETIC MRI VOLUMES....16
3.5.2 SYNTHETIC VEP WAVEFORMS....17
3.5.3 DATASET BEHAVIOUR....18
3.6 DATA INTERFACE....18
3.7 MRI PREPROCESSING....19
3.8 VEP PREPROCESSING....20
3.9 EVALUATION STRATEGY....22
CHAPTER 4 SYSTEM DESIGN AND IMPLEMENTATION....23
4.1 ARCHITECTURE OVERVIEW....23
4.2 SPATIAL ENCODER (3D RESIDUAL CNN)....24
4.3 TEMPORAL ENCODER (1D CNN)....24
4.4 CROSS-ATTENTION FUSION MODULE....25
4.5 CLASSIFICATION HEADS AND MULTI-TASK LOSS....26
4.6 TRAINING PROCEDURE....27
4.7 EXPLAINABILITY....28
4.7.1 3D GRAD-CAM ON THE FUSED LOGIT....28
4.7.2 CROSS-ATTENTION HEATMAP....29
4.8 SOFTWARE STRUCTURE AND CONFIGURATION....29
4.9 TEST DESIGN....30
CHAPTER 5 RESULTS....32
5.1 INTRODUCTION....32
5.2 SIGNAL CONDITIONING RESULTS....32
5.2.1 FREQUENCY RESPONSE....32
5.2.2 MAINS-INTERFERENCE ATTENUATION....33
5.2.3 P100 LATENCY....34
5.2.4 CONDITIONED-WAVEFORM STATISTICS....35
5.2.5 NOMINAL TEMPORAL-TOKEN BINS....36
5.3 SYNTHETIC MRI CHARACTERISATION....36
5.4 MODEL COMPLEXITY....36
5.5 INTERFACE AND SHAPE VERIFICATION....37
5.6 SOFTWARE VERIFICATION RESULTS....37
5.7 TRAINING ACCEPTANCE CRITERIA....39
5.8 SUMMARY OF RESULTS....40
CHAPTER 6 DISCUSSION....41
6.1 INTRODUCTION....41
6.2 INTERPRETATION OF THE RESULTS....41
6.2.1 SIGNAL CONDITIONING....41
6.2.2 SYNTHETIC DATA AND THE FOREGROUND MASK....42
6.2.3 MODEL SIZE AND COST....42
6.2.4 VERIFICATION....43
6.3 ASSESSMENT AGAINST THE OBJECTIVES, QUESTIONS AND HYPOTHESES....43
6.4 DESIGN OBSERVATIONS....44
6.4.1 COARSE TEMPORAL TOKENS....44

6.4.2 SYNTHETIC SEPARABILITY....44
6.4.3 MULTI-TASK SUPERVISION AND ATTENTION....44
6.5 COMPARISON WITH THE LITERATURE....45
6.6 CONSTRAINTS AND THREATS TO VALIDITY....45
CHAPTER 7 CONCLUSION, RECOMMENDATIONS AND FUTURE WORK 47
7.1 CONCLUSION....47
7.2 CONCLUSIONS AGAINST THE OBJECTIVES....47
7.3 RECOMMENDATIONS....48
7.4 FUTURE WORK....49
7.5 CLOSING REMARKS....49
REFERENCES....51
APPENDIX 1 DEFAULT CONFIGURATION....56
A1.1 CONFIGURATION PARAMETERS....56
APPENDIX 2 SOURCE CODE LISTINGS....57
A2.1 OVERVIEW....57
A2.2 LISTINGS....57
APPENDIX 3 TEST INVENTORY....73
A3.1 TEST FILES....73
A3.2 TEST FUNCTIONS....73
APPENDIX 4 REPRODUCTION NOTES....76
A4.1 SOFTWARE ENVIRONMENT....76
A4.2 COMMANDS....76
A4.3 MEASUREMENTS REPORTED IN CHAPTER 5....76
A4.4 ITEMS NOT REPRODUCED HERE....76

LIST OF TABLES

Table 1.1 Organisation of the report.....6

Table 2.1 Clinical conditions considered and the way each is encoded in the synthetic data.....8

Table 2.2 Canonical fusion strategies and the position of this project.....10

Table 2.3 Key references and where each is used in the project.....12

Table 3.1 Tensor contract at the default configuration.....14

Table 3.2 Software stack. The last three libraries are listed in `requirements.txt` but are not imported by the current source or tests.....15

Table 3.3 Public data resources identified in the project blueprint (descriptions as stated there; not independently re-verified for this report).....16

Table 3.4 Class-conditioned parameters of the synthetic generators.....17

Table 3.5 MRI preprocessing parameters (`MRIConfig`).....20

Table 3.6 VEP preprocessing parameters (`VEPConfig`).....21

Table 3.7 Evaluation strategy: what is measured, how, and which objective it supports.....22

Table 4.1 Spatial encoder layers (batch size 1) and trainable parameters.....24

Table 4.2 Temporal encoder layers (batch size 1) and trainable parameters.....25

Table 4.3 Training configuration (`TrainConfig`) and model configuration (`ModelConfig`) at the default values.....28

Table 4.4 Source modules, line counts and responsibilities.....30

Table 4.5 Test files and what they verify.....31

Table 5.1 Effective magnitude response (dB) at selected frequencies.....33

Table 5.2 Detected P100 latency (ms; mean ± SD over 50 trials per class) and mean error against the ground truth.....34

Table 5.3 Group delay of the single-pass (causal) implementation of the same filters.....35

Table 5.4 Synthetic MRI: mean intensity of regions of interest and effect of the 60th-percentile foreground mask.....36

Table 5.5 Trainable parameters and multiply-accumulates (millions) per sample at the default configuration.....36

Table 5.6 Statement coverage by module (reconstructed from the recorded coverage data).....38

Table 5.7 Training-loop acceptance criteria asserted by `tests/integration/test_training_loop.py`.....39

Table 5.8 Summary of the objective results.....40

Table 6.1 Assessment of research questions and hypotheses against the evidence. 43
Table 6.2 Constraints of the study and their consequence for the conclusions. . . 45
Table 7.1 Status of each objective 47
Table 7.2 Prioritised recommendations 48
Table A1.1 Default configuration values (`src/config.py`) 56
Table A2.1 Source files reproduced in this appendix 57
Table A3.1 Test files, number of test functions and the component each exercises. 73

LIST OF FIGURES

Figure 1.1 Conceptual motivation: MRI answers *where*, the VEP answers *when*, and cross-attention fusion links the two while producing material that can be inspected.....3

Figure 3.1 End-to-end data flow of the proposed framework. Spatial tokens act as queries (Q); temporal tokens act as keys and values (K, V). Dashed arrow: explainability outputs.....14

Figure 3.2 Axial slices of synthetic MRI volumes generated by the project's own generator (seeded). Dashed boxes mark the class-specific region: lesion proxy (Optic neuritis / CVI, index 20) and atrophy proxy (Glaucoma, index 60).....17

Figure 3.3 Synthetic VEP waveforms for the three classes, before (left) and after (right) the project's conditioning function `clean_vep_signal`. Dotted line: ground-truth P100 latency.....18

Figure 3.4 The two dataset classes share one preprocessing path. The manifest-driven class is intended for a future labelled cohort; the in-memory synthetic class is used in the current prototype.....19

Figure 3.5 Preprocessing chains for the MRI stream (top) and the VEP stream (bottom), with the function responsible for each step.....19

Figure 4.1 CAFN architecture with tensor shapes (default configuration). The two auxiliary heads read the mean-pooled unimodal tokens; the fused head reads the mean-pooled fused tokens.....23

Figure 4.2 Nominal mapping of the 16 temporal tokens onto the 2 s epoch (125 ms per token) over an example conditioned VEP. The shaded region is token 0.....25

Figure 4.3 Structure of the cross-attention fusion module (CAFM). The attention-weight tensor has shape (B, 4, 64, 16) and is returned for inspection..26

Figure 4.4 Training and evaluation flow. A seeded permutation splits the data; each epoch is followed by validation; the trained model can be checkpointed and passed to the explainability tools.....27

Figure 4.5 Generation of the two explanation artefacts: the Grad-CAM map (A) and the cross-attention heatmap (B).....29

Figure 4.6 Module structure and dependencies of the code base.....30

Figure 5.1 Magnitude response of the conditioning filters in the effective (forward-backward) form.....33

Figure 5.2 Amplitude spectrum of one synthetic VEP before and after conditioning. The dotted line marks 50 Hz.....34

Figure 5.3 Left: group delay of the single-pass filters versus the zero-phase pipeline (shaded band: ±1 sample, 4 ms). Right: mean P100 latency error by class (shaded band: ±4 ms).....35

Figure 5.4 Distributions over 50 synthetic trials per class: detected P100 latency after conditioning (left) and peak value after z-scoring (right).....35

Figure 5.5 Left: trainable parameters by component. Right: multiply-accumulate operations per sample by component group.....37

Figure 5.6 Left: number of test functions per test file (blue: unit; green: integration). Right: statement coverage per module (red: below 90%). 38

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

1D / 3D	One- / three-dimensional
BN	Batch normalisation
CAFN	Cross-Attention Fusion Network (the model developed in this project)
CNN	Convolutional neural network
CSV	Comma-separated values (file format)
CVI	Cortical (cerebral) visual impairment
EEG	Electroencephalography
ERG	Electroretinography
FFN	Position-wise feed-forward network
Grad-CAM	Gradient-weighted class activation mapping
HCP	Human Connectome Project
IIR	Infinite impulse response (filter)
ISCEV	International Society for Clinical Electrophysiology of Vision
LN	Layer normalisation
MAC	Multiply–accumulate operation
MONAI	Medical Open Network for AI (software framework)
MRI / sMRI	(Structural) magnetic resonance imaging
N75, P100, N135	Components of the pattern-reversal VEP: negative peak near 75 ms, positive peak near 100 ms, negative peak near 135 ms
NIfTI	Neuroimaging Informatics Technology Initiative (file format)
OASIS	Open Access Series of Imaging Studies
RAS	Right–anterior–superior (canonical image orientation)
ReLU	Rectified linear unit
ROI	Region of interest
SD	Standard deviation
VEP	Visual evoked potential
A	Amplitude of the synthetic P100 component (Equation 3.1)
τ	Latency of the P100 component; also the percentile threshold τ_p in Equation 3.3
$\mathbf{d_{model}}$	Width of the token embedding (128 by default)
H, $\mathbf{d_h}$	Number of attention heads (4) and dimension per head
$\mathbf{X_s}$**,** $\mathbf{X_t}$	Spatial and temporal token matrices
Q, K, V	Query, key and value matrices of the cross-attention module
$\mathbf{W_Q}$**,** $\mathbf{W_K}$**,** $\mathbf{W_V}$**,** $\mathbf{W_O}$	Learned projection matrices of the cross-attention module
$\mathbf{A_h}$**,** $\mathbf{O_h}$	Attention weights and output of head h

$\mathbf{L_f, L_s, L_t}$	Cross-entropy losses of the fused, spatial and temporal heads
$\boldsymbol{\lambda}$	Weight of the auxiliary losses (0.3)
$\boldsymbol{\mu, \sigma, \varepsilon}$	Mean, standard deviation and stabilising constant of z-score normalisation
$\boldsymbol{\alpha_k}$, **M**	Grad-CAM channel weights and resulting activation map

Chapter 1
Introduction

1.1 BACKGROUND

Vision depends on an uninterrupted chain of structures: the retina, the optic nerve, the optic chiasm, the lateral geniculate nucleus, the optic radiations and, finally, the primary and associated visual cortices (areas V1, V2 and V3). A lesion at any point of this chain can degrade sight, and the clinical picture depends on where along the pathway the damage lies and on whether it disturbs the anatomy, the conduction of the electrical signal, or both.

Three conditions illustrate the range. Glaucoma is a progressive optic neuropathy; a systematic review and meta-analysis estimated its global prevalence at 3.54% among people aged 40 to 80 years, corresponding to about 64.3 million people in 2013 and a projected 111.8 million by 2040 (Tham et al., 2014). Histological work has shown that the degeneration in human glaucoma is not confined to the eye but extends to the intracranial optic nerve, the lateral geniculate nucleus and the visual cortex (Gupta et al., 2006). Optic neuritis is an inflammatory, usually demyelinating, disorder of the optic nerve whose electrophysiological hallmark is a delayed P100 peak of the visual evoked potential (Halliday et al., 1972). Cortical or cerebral visual impairment (CVI) denotes reduced vision that arises from damage to the brain's visual pathways rather than the eye; the literature uses a variety of terms and definitions for it (Sakki et al., 2018).

Two investigations are routinely used to examine this pathway, and they answer different questions. Structural magnetic resonance imaging (sMRI) provides a three-dimensional picture of anatomy, so that atrophy and focal lesions can be localised. The visual evoked potential (VEP) is a one-dimensional electrophysiological recording of the cortical response to a visual stimulus; its latency and amplitude, in particular those of the P100 component, reflect the integrity and speed of conduction along the visual pathway, and its recording and interpretation are standardised by the International Society for Clinical Electrophysiology of Vision (ISCEV) (Odom et al., 2016). Neither investigation alone tells the whole story: MRI carries no timing information and the VEP carries no spatial localisation.

Multimodal data fusion has long been proposed as a way to exploit such complementary information (Calhoun & Sui, 2016). Deep learning has made it possible to learn fusion functions directly from data, and the attention mechanism introduced for sequence modelling (Vaswani et al., 2017) offers a particularly flexible way to let one stream of features select what it needs from another. This project investigates that idea for the pairing of 3D structural MRI and 1D visual evoked potentials.

1.2 MOTIVATION AND CHALLENGES

Automated, data-driven support for the diagnosis and classification of visual-pathway disorders faces three principal bottlenecks. They motivate the design choices described in the rest of this report.

1.2.1 UNIMODAL ISOLATION

Most computational pipelines analyse either imaging or electrophysiology, not both. A model that only sees the MRI volume can detect structural change but is blind to a conduction delay that has not yet produced visible atrophy; a model that only sees the VEP can detect the delay but cannot say where in the pathway it originates.

1.2.2 CROSS-MODAL HETEROGENEITY

A volumetric MRI array of shape (1, D, H, W) and a waveform of shape (1, L) differ in dimensionality, sampling, noise structure and units. Simple fusion strategies, such as concatenating pooled features or averaging unimodal predictions, give every feature the same standing and cannot express which spatial feature is relevant to which temporal feature. Subtle pathology markers can be diluted as a result.

1.2.3 LIMITED INTERPRETABILITY

A black-box classifier is difficult to trust in a clinical setting. A clinician needs to see which parts of the image and which parts of the waveform drove a decision, so that the output can be checked against clinical knowledge.

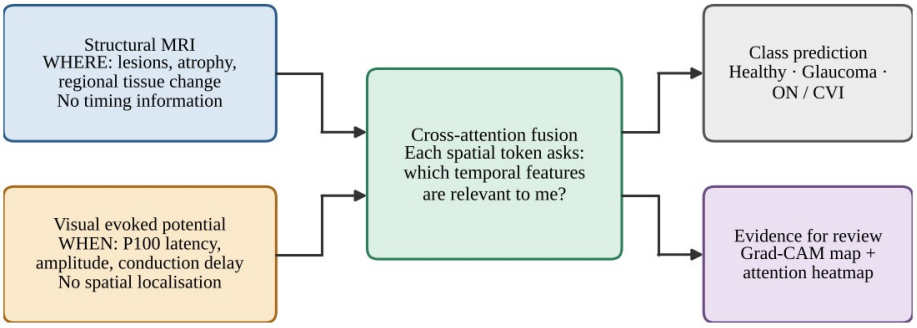


Figure 1.1: Conceptual motivation: MRI answers *where*, the VEP answers *when*, and cross-attention fusion links the two while producing material that can be inspected.

1.3 PROBLEM STATEMENT

To design, implement and verify an end-to-end deep-learning framework that ingests a paired 3D structural MRI volume and a 1D visual evoked potential, fuses them with an attention mechanism that lets spatial features query temporal features, classifies the pair into Healthy Control, Glaucoma or Optic Neuritis / Cortical Visual Impairment, and exposes spatial and spatial–temporal explanations of each decision.

The problem is therefore both a modelling problem (how to relate two incompatible feature spaces) and an engineering problem (how to build a pipeline whose interfaces, preprocessing and training behaviour can be tested before a clinical cohort is available).

1.4 OBJECTIVES AND RESEARCH QUESTIONS

1.4.1 OBJECTIVES

The project pursues the following objectives. They are revisited, one by one, in the conclusions (Chapter 7).

1. **O1: Data interface.** Provide a single dataset abstraction that reads paired MRI and VEP records from a manifest and standardises them to fixed-shape tensors ($1 \times 64 \times 64 \times 64$ and 1×500).
2. **O2: Signal conditioning.** Implement MRI standardisation (RAS orientation, 1 mm isotropic resampling, foreground masking, intensity scaling) and VEP conditioning (zero-phase 1–100 Hz band-pass, 50 Hz notch, resampling, z-score) so that clinically meaningful timing is not distorted.

3. **O3: Dual encoders.** Build a 3D residual CNN and a 1D CNN that map the two modalities to 64 spatial and 16 temporal tokens in a shared 128-dimensional space.
4. **O4: Cross-attention fusion.** Implement multi-head cross-attention in which spatial tokens are queries and temporal tokens are keys and values, and train it with a fused head plus two auxiliary unimodal heads (multi-task supervision).
5. **O5: Explainability.** Provide 3D Grad-CAM activation maps and a spatial–temporal cross-attention heatmap.
6. **O6: Verification.** Verify interfaces and behaviour with an automated unit and integration test suite, and quantify what can be quantified without clinical data (filter behaviour, model size, test coverage).
7. **O7: Honest scoping.** State clearly what has and has not been demonstrated, and define the path from prototype to clinically credible evaluation.

1.4.2 RESEARCH QUESTIONS

1. **RQ1.** Can paired 3D MRI and 1D VEP data be brought into a common token representation through one manifest-driven pipeline?
2. **RQ2.** Does the conditioning pipeline preserve the P100 latency, the landmark that carries diagnostic meaning in the VEP?
3. **RQ3.** Can the cross-attention fusion network be optimised end-to-end under multi-task supervision, and what is its computational footprint?
4. **RQ4.** Can the system emit explanation artefacts (spatial maps and attention matrices) from the same forward pass that produces the prediction?

1.4.3 WORKING HYPOTHESES

Because no labelled clinical data are available, the hypotheses are engineering hypotheses that can be tested on synthetic data: (H1) zero-phase filtering keeps the detected P100 latency within one sample period (4 ms at 250 Hz) of the ground truth; (H2) the multi-task training loop reduces training loss substantially on a class-conditioned synthetic task and exceeds chance-level validation accuracy; and (H3) all modules respect a single shape contract so that the preprocessing, encoders, fusion module and heads cannot silently disagree.

1.5 SCOPE OF THE PROJECT

In scope. The project covers (i) the data-loading and preprocessing layer for MRI (NIfTI) and VEP (CSV) files; (ii) a synthetic, class-conditioned data generator that exposes the same interface as real data; (iii) the CAFN model (two encoders, a cross-attention fusion module and three classification heads); (iv) a multi-task training loop with checkpointing; (v) the explainability tools; and (vi) an automated test suite.

Out of scope. The project does not include clinical data acquisition, expert labelling, clinical-grade brain extraction, anatomical registration of features to named cortical areas, prospective clinical validation or regulatory work. The command-line trainer is wired only to the synthetic dataset; the real-data dataset class is available through the Python interface.

1.6 LIMITATIONS OF THE STUDY

- **No labelled clinical cohort.** The public repositories surveyed for the project provide structural or retinotopic scans and generic electrophysiology but not the Healthy / Glaucoma / Optic Neuritis-CVI diagnosis labels required for training. All learning experiments are therefore confined to synthetic data, which are deliberately simple and cannot support any claim about clinical performance.
- **Prototype preprocessing.** Foreground masking is a percentile threshold, not validated skull-stripping, and no anatomical registration is carried through to the model.
- **Learned, uncalibrated tokens.** Spatial and temporal tokens are learned feature positions, not anatomically registered brain regions or calibrated millisecond bins.
- **No external baselines or uncertainty estimates.** The repository contains no unimodal or concatenation baseline comparison, no missing-modality handling, and no calibration.

1.7 CONTRIBUTIONS AND DISTINCTION FROM EXISTING WORK

It is important to separate what this project contributes from what it reuses. The *reused* building blocks are standard: scaled dot-product and multi-head attention (Vaswani et al., 2017), residual convolutional blocks (He et al., 2016), Grad-CAM (Selvaraju et al., 2017), Butterworth and notch filtering through SciPy (Virtanen et al.,

2020), and MONAI transforms for volumetric resampling and cropping (Cardoso et al., 2022). The *contribution* of the project is their integration into one tested, configuration-driven pipeline for the specific pairing of 3D MRI and 1D VEP:

- a shared shape contract (a single configuration module) enforced across preprocessing, both encoders, the fusion module and the training loop;
- a cross-attention fusion design in which spatial tokens query temporal tokens, with attention weights returned for inspection;
- a multi-task objective with auxiliary unimodal heads to discourage collapse onto one modality;
- a class-conditioned synthetic data generator and an end-to-end test suite that exercise the real file path (NIfTI and CSV on disk) through to an optimiser step and the explainability outputs; and
- a transparent record of limitations and of the steps required before clinical evaluation.

1.8 ORGANISATION OF THE REPORT

Table 1.1: Organisation of the report.

Chapter	Content
1 Introduction	Background, motivation, problem statement, objectives, scope and limitations.
2 Literature Review	Clinical background, deep learning for MRI and electrophysiology, multimodal fusion, attention, explainability, public data resources and the research gap.
3 Methodology	Overall approach, tools, data sources, synthetic data, data interface, MRI and VEP preprocessing, and the evaluation strategy.
4 System Design and Implementation	CAFN architecture, cross-attention fusion, multi-task loss, training, explainability, software structure and test design.
5 Results	Objective presentation of the measurements: filter behaviour, synthetic data statistics, model size, test results and coverage, and training acceptance criteria.
6 Discussion	Interpretation of the results, design observations, comparison with the literature and threats to validity.
7 Conclusion, Recommendations and Future Work	Conclusions against the objectives, prioritised recommendations and future work.
	References (APA) and Appendices (configuration, source listings, test inventory, reproduction notes).

Chapter 2
Literature Review

2.1 INTRODUCTION

This chapter summarises the research and technical background on which the project rests. It first describes the clinical conditions and the two investigations considered, then reviews deep learning for volumetric imaging and for one-dimensional signals, multimodal fusion strategies, attention and cross-attention, explainability, and the public data resources that were surveyed. It closes by stating the gap that the project addresses and the position it takes.

2.2 CLINICAL BACKGROUND

2.2.1 GLAUCOMA, OPTIC NEURITIS AND CORTICAL VISUAL IMPAIRMENT

Glaucoma is a progressive optic neuropathy and a leading cause of irreversible visual loss; its burden is large and growing (Tham et al., 2014). Although it is usually described through damage to the optic nerve head, post-mortem evidence indicates that degeneration also affects the intracranial optic nerve, the lateral geniculate nucleus and the visual cortex (Gupta et al., 2006). Electrophysiological abnormalities have been reported in ocular hypertension and glaucoma with pattern electroretinography and visual evoked potentials (Parisi et al., 2006). The synthetic data used in this project encode glaucoma by a focal loss of MRI intensity (an atrophy proxy) and a reduced VEP amplitude with near-normal latency.

Optic neuritis is inflammation of the optic nerve. Its classical electrophysiological signature is a prolonged P100 latency of the pattern-reversal VEP with a relatively preserved waveform (Halliday et al., 1972), and the lesion is typically visible on MRI as a focal signal abnormality. Cortical or cerebral visual impairment arises from damage to the posterior visual pathways and visual cortex; the terminology and definitions used for it are not uniform across the literature (Sakki et al., 2018). In this project, optic neuritis and CVI are grouped into a single class and encoded synthetically by a focal MRI hyperintensity (a lesion proxy) and a delayed P100.

Table 2.1: Clinical conditions considered and the way each is encoded in the synthetic data.

Condition	Structural (MRI) signature	Electrophysiological (VEP) signature	Synthetic encoding in this project
Healthy control	No focal abnormality	P100 near 100 ms with normal amplitude	Background-like volume; P100 at 100 ms, amplitude 6.0
Glaucoma	Atrophy along the visual pathway (Gupta et al., 2006)	Amplitude and/or latency abnormalities reported (Parisi et al., 2006)	Focal intensity reduction (×0.4); P100 at 108 ms, amplitude 2.5
Optic neuritis / CVI	Focal lesion; posterior-pathway damage in CVI	Delayed P100 (Halliday et al., 1972)	Focal hyperintensity (×1.8); P100 at 140 ms, amplitude 4.0

2.2.2 THE VISUAL EVOKED POTENTIAL AND THE P100 COMPONENT

The VEP is the scalp-recorded cortical response to a visual stimulus, most commonly a pattern reversal. In the ISCEV standard, the principal landmarks of the pattern-reversal response are a negative N75, a positive P100 and a negative N135 component, and the timing of the P100 is the key quantitative measure (Odom et al., 2016). Because a clinically significant delay may amount to a few tens of milliseconds, any signal-processing step applied before analysis must not shift the waveform in time. This requirement is the reason the project uses zero-phase filtering, discussed in Section 3.8.

2.3 DEEP LEARNING FOR VOLUMETRIC NEUROIMAGING

Convolutional neural networks extended to three dimensions are the standard way to learn directly from volumetric scans. Efficient multi-scale 3D CNNs were developed for brain-lesion segmentation (Kamnitsas et al., 2017), and the 3D U-Net demonstrated dense volumetric segmentation from sparse annotation (Çiçek et al., 2016). Residual connections, which make deep networks easier to optimise (He et al., 2016), are routinely carried over to the 3D setting. The spatial encoder of this project follows that tradition: a small 3D residual network that reduces a $64 \times 64 \times 64$ volume to a $4 \times 4 \times 4$ grid of 128-dimensional features.

Volumetric pipelines depend on preprocessing. Brain extraction (skull-stripping) is a prerequisite for many analyses, and dedicated tools exist: HD-BET, a neural-network brain extractor for multi-sequence MRI (Isensee et al., 2019), and SynthStrip, a skull-stripping method designed to work across image types (Hoopes et al., 2022). The MONAI framework provides open-source, domain-specific transforms

and training components for medical imaging (Cardoso et al., 2022), and nibabel provides access to NIfTI files and their orientation metadata (NiBabel Developers, n.d.). The present project uses nibabel and MONAI for loading, orientation, resampling and cropping, and replaces dedicated skull-stripping with a lightweight intensity-percentile mask that is explicitly labelled a placeholder (Section 3.7).

2.4 DEEP LEARNING FOR ONE-DIMENSIONAL ELECTROPHYSIOLOGICAL SIGNALS

One-dimensional convolutional networks learn local temporal patterns, such as the rise and fall of a waveform component, with learned filters; stacked strided convolutions then expand the receptive field. Before learning, evoked-potential recordings are conventionally conditioned to remove baseline drift, muscle (EMG) noise and mains interference. A Butterworth filter has a maximally flat pass-band (Butterworth, 1930), and a narrow IIR notch removes the mains frequency; applying either filter forward and backward over the record (as implemented by SciPy's `sosfiltfilt` and `filtfilt`, following Gustafsson, 1996) cancels the phase response so that waveform timing is unchanged (Oppenheim & Schafer, 2010). This project adopts exactly this conditioning chain for the VEP and a four-stage 1D CNN as the temporal encoder.

2.5 MULTIMODAL DATA FUSION

Calhoun and Sui (2016) argued that combining brain-imaging modalities can reveal relationships invisible to any single modality because each modality carries complementary information. Surveys of multimodal machine learning (Baltrušaitis et al., 2019) and of deep multimodal learning (Ramachandram & Taylor, 2017) distinguish where in the processing chain the modalities are combined. Table 2.2 summarises the three canonical strategies and the position of the present work.

Table 2.2: Canonical fusion strategies and the position of this project.

Strategy	How modalities are combined	Advantages	Disadvantages for MRI + VEP
Early (input-level)	Raw or minimally processed inputs are joined before learning	Simple; allows low-level interaction	A 3D volume and a 1D waveform have no natural common input space; severe misalignment
Late (decision-level)	Unimodal predictions are averaged or voted	Modular; tolerant of a missing modality	No interaction between modalities before the decision; subtle cross-modal evidence is lost
Intermediate (feature-level)	Learned features are combined, by concatenation, gating or attention	Learns task-specific interactions	Concatenation weights all features equally; attention is needed to express relevance
This project	Intermediate fusion by cross-attention: spatial tokens query temporal tokens, plus auxiliary unimodal heads	Selective, inspectable interaction; unimodal supervision limits modality collapse	Requires paired data; learned tokens are not calibrated anatomy or time

2.6 ATTENTION AND CROSS-ATTENTION

The Transformer replaced recurrence with scaled dot-product attention, in which queries are compared with keys and the resulting weights are used to average values (Vaswani et al., 2017). Attention is *self*-attention when queries, keys and values come from one sequence and *cross*-attention when the queries come from one sequence and the keys and values from another. Vision Transformers showed that images can be treated as sequences of patch tokens (Dosovitskiy et al., 2021), which is the same device used here to turn a volume into 64 spatial tokens. Cross-modal attention has been used to relate modalities that are not time-aligned: the multimodal transformer lets one modality attend to another without requiring explicit alignment (Tsai et al., 2019), and ViLBERT uses co-attentional layers to exchange information between visual and linguistic streams (Lu et al., 2019).

The central design choice of this project follows from that line of work. Rather than flattening and concatenating the two modalities, each spatial token is treated as a *query* that asks which temporal tokens are relevant to it. The softmax over temporal tokens yields an attention matrix that can be inspected.

Two normalisation layers that appear in the model are also drawn from the literature: batch normalisation in the convolutional encoders (Ioffe & Szegedy, 2015)

and layer normalisation in the fusion block (Ba et al., 2016). The optimiser is Adam (Kingma & Ba, 2015).

2.7 EXPLAINABILITY

Gradient-weighted class activation mapping (Grad-CAM) uses the gradients of a class score with respect to the activations of a convolutional layer to produce a coarse localisation map (Selvaraju et al., 2017). The project applies the same idea in three dimensions, starting the back-propagation from the fused logit so that the map reflects the multimodal decision.

The literature also warns against over-interpreting explanation artefacts. Attention weights are not automatically faithful explanations of a model's output (Jain & Wallace, 2019), although whether they can serve as explanations depends on how that question is defined and tested (Wiegreffe & Pinter, 2019). Saliency methods can also produce visually plausible maps that are insensitive to the model's learned parameters, which motivates sanity checks such as randomising weights (Adebayo et al., 2018). These cautions shape how the outputs of this project are described: as inspection aids, not as clinical evidence.

2.8 PUBLIC DATA RESOURCES AND THE LABELLING GAP

Several public repositories were identified in the project blueprint as sources of structural and functional neuroimaging and of electrophysiology: OpenNeuro datasets (the project lists ds003787 and ds004496) (OpenNeuro, n.d.-a, n.d.-b), the Human Connectome Project including its 7 T retinotopy data (Van Essen et al., 2013; Benson et al., 2018), the OASIS-3 database (LaMontagne et al., 2019), the MNE-Python sample dataset (Gramfort et al., 2013) and PhysioNet (Goldberger et al., 2000). They are valuable for developing preprocessing and for retinotopic mapping of the visual cortex, but, as the project blueprint itself notes, none of them provides the Healthy / Glaucoma / Optic Neuritis-CVI diagnosis labels that this model targets. A labelled, paired clinical cohort, or a clinical partner to label a subset, is still needed before real training is possible. This labelling gap is the reason for the synthetic data generator described in Section 3.5.

2.9 RESEARCH GAP AND POSITION OF THIS PROJECT

The literature reviewed above establishes the ingredients: complementary information in MRI and VEP, 3D and 1D convolutional encoders, cross-modal attention and Grad-CAM. What is missing from an engineering perspective is a transparent, tested, configuration-driven pipeline that connects them for the specific pairing of 3D structural MRI and 1D VEP, and that can be exercised end-to-end before a labelled clinical cohort exists. The project fills that gap at the level of a *research prototype*. It does not claim a new learning algorithm, and no clinical performance claim is made.

Table 2.3: Key references and where each is used in the project.

Reference	Contribution	Use in this project
Vaswani et al. (2017)	Scaled dot-product and multi-head attention	Cross-attention fusion module (Section 4.4)
He et al. (2016)	Residual learning	3D residual blocks of the spatial encoder (Section 4.2)
Selvaraju et al. (2017)	Grad-CAM	3D Grad-CAM on the fused logit (Section 4.7)
Odom et al. (2016)	ISCEV standard for clinical VEP	P100 as the timing landmark; epoch and filter choices (Section 3.8)
Butterworth (1930); Gustafsson (1996)	Maximally flat filter; forward-backward filtering	Zero-phase 1–100 Hz band-pass (Section 3.8)
Cardoso et al. (2022)	MONAI medical imaging framework	Resize, ScaleIntensity, ResizeWithPadOrCrop (Section 3.7)
Paszke et al. (2019)	PyTorch	All neural-network modules and training
Kingma & Ba (2015)	Adam optimiser	Training loop (Section 4.6)
Jain & Wallace (2019); Adebayo et al. (2018)	Limits of attention and saliency as explanations	Interpretation of explainability outputs (Chapter 6)

Chapter 3
Methodology

3.1 OVERVIEW OF THE APPROACH

The project follows an iterative engineering methodology in which the interfaces are fixed first and each stage is then implemented and tested against them. Because no labelled clinical data were available, the methodology deliberately separates *software correctness* (which can be established now) from *clinical validity* (which cannot). The work proceeded in the following steps.

1. **Define the shape contract.** All tensor shapes, sampling parameters and hyper-parameters are fixed in a single configuration module before any model code is written (Section 4.8).
2. **Specify the data interface.** A manifest of paired records (subject, MRI path, VEP path, sampling rate, label) drives a PyTorch dataset that returns standardised tensors (Section 3.6).
3. **Implement preprocessing.** MRI standardisation and VEP conditioning are implemented as separate, validated, side-effect-free functions (Sections 3.7 and 3.8).
4. **Generate class-conditioned synthetic data.** Because labelled data are missing, synthetic volumes and waveforms with the same interface as real data are produced so that the entire pipeline can be exercised (Section 3.5).
5. **Build the model.** A 3D residual spatial encoder, a 1D temporal encoder, a cross-attention fusion module and three classification heads are implemented (Chapter 4).
6. **Train with multi-task supervision.** A fused loss and two auxiliary unimodal losses are optimised jointly (Section 4.6).
7. **Add explainability.** 3D Grad-CAM and the cross-attention heatmap are generated from the trained model (Section 4.7).
8. **Verify.** A unit and integration test suite, plus measurements on the synthetic data, establish what the prototype does and does not demonstrate (Section 3.9 and Chapter 5).

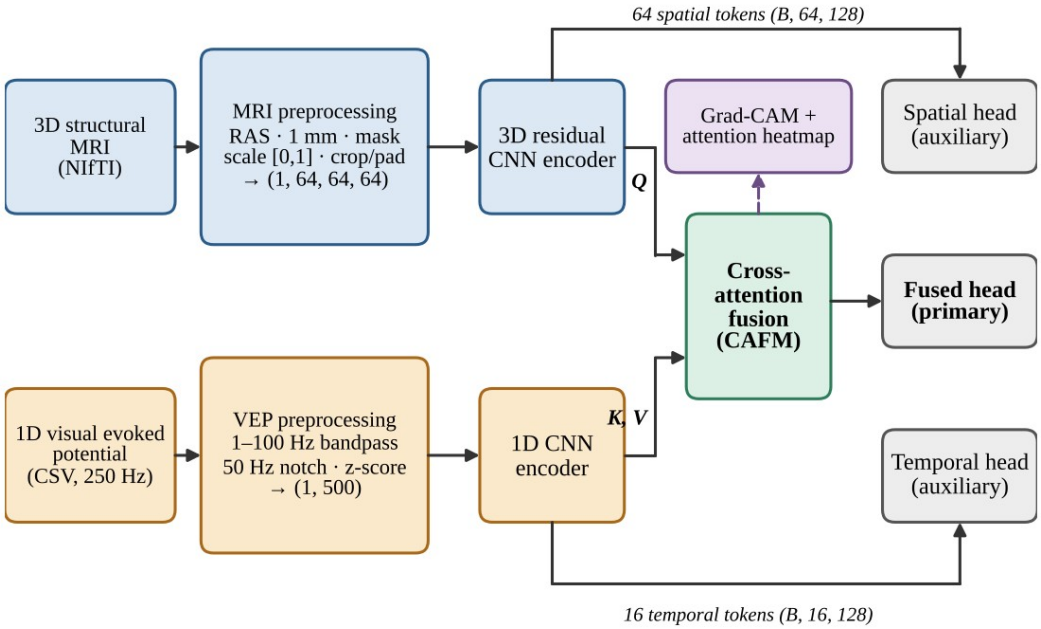


Figure 3.1: End-to-end data flow of the proposed framework. Spatial tokens act as queries (Q); temporal tokens act as keys and values (K, V). Dashed arrow: explainability outputs.

At the default configuration a paired sample moves through the tensor contract of Table 3.1. The batch dimension *B* is added by the data loader; the channel dimension is part of each sample.

Table 3.1: Tensor contract at the default configuration.

Stage	MRI stream	VEP stream
Raw / loaded	3D array from NIfTI	1D numeric waveform
After preprocessing	(1, 64, 64, 64)	(1, 500)
Encoder output (tokens)	(B, 64, 128)	(B, 16, 128)
Cross-attention weights	(B, 4, 64, 16): 4 heads × 64 spatial queries × 16 temporal keys	
Fused tokens	(B, 64, 128)	n/a
Logits	Fused (B, 3); spatial auxiliary (B, 3)	Temporal auxiliary (B, 3)

3.2 DESIGN PRINCIPLES

- **Single source of truth.** Every shape and hyper-parameter lives in frozen dataclasses in `src/config.py`; the preprocessing stream, both encoders, the fusion module and the training loop read from it, so they cannot silently disagree.
- **Fail loudly on invalid input.** Preprocessing functions validate dimensionality, finiteness, sampling limits (for example, a band-pass upper edge above the Nyquist frequency is rejected) and signal length, and raise explicit errors.

- **Same interface for real and synthetic data.** The synthetic generator writes real NIfTI and CSV files and a manifest, so the exact code path that a clinical cohort would use is tested.
- **Replaceable placeholders.** Components known to be prototype-grade, notably the foreground mask, are isolated behind one function so a validated tool can be substituted without changing anything downstream.
- **Test every module and the whole system.** Each module has unit tests; two integration suites test the raw-file-to-optimiser-step path and the training loop.

3.3 TOOLS AND ENVIRONMENT

Table 3.2: Software stack. The last three libraries are listed in `requirements.txt` but are not imported by the current source or tests.

Component	Role in the project
Python 3	Implementation language (the exact version is not recorded in the repository)
PyTorch (Paszke et al., 2019)	Tensors, neural-network modules, automatic differentiation, optimisation, data loading; default training device is the CPU
MONAI (Cardoso et al., 2022)	`Resize` (trilinear), `ScaleIntensity`, `ResizeWithPadOrCrop` for the MRI stream
NiBabel (NiBabel Developers, n.d.)	NIfTI reading and writing; reorientation to the closest canonical (RAS) frame
SciPy (Virtanen et al., 2020)	`butter`, `sosfiltfilt`, `iirnotch`, `filtfilt` for VEP conditioning
NumPy (Harris et al., 2020)	Array computation; synthetic data generation
Matplotlib (Hunter, 2007)	Grad-CAM slice overlays and the attention heatmap
pytest, pytest-cov	Unit and integration tests and coverage measurement
nilearn, MNE-Python (Gramfort et al., 2013), scikit-learn (Pedregosa et al., 2011)	Listed as dependencies for data access and future evaluation; not yet used

The source code is held in a Git repository (*NeuroImaging*). It comprises 1,054 lines of source in twelve modules and 1,004 lines of tests in twelve test files, plus a configuration module and test fixtures.

3.4 DATA SOURCES

The project blueprint identifies the public repositories in Table 3.3. They are useful for developing and testing preprocessing, and for retinotopic mapping of visual cortex, but none carries the diagnostic labels required for supervised training.

Table 3.3: Public data resources identified in the project blueprint (descriptions as stated there; not independently re-verified for this report).

Modality	Resource	Stated utility
3D sMRI / fMRI	OpenNeuro ds004496	High-resolution visual cortex mapping under retinotopic stimulation
3D sMRI / fMRI	OpenNeuro ds003787	Multimodal structural and functional visual-pathway benchmarks
3D sMRI / fMRI	Human Connectome Project 7 T retinotopy (Van Essen et al., 2013; Benson et al., 2018)	7 T structural and retinotopic functional scans of cortical visual areas V1, V2, V3
3D sMRI	OASIS (LaMontagne et al., 2019)	Large-scale structural MRI for normalisation and grey-matter segmentation baselines
1D VEP / EEG	MNE-Python sample dataset (Gramfort et al., 2013)	Visual and auditory stimulus responses on occipital sensors
1D VEP / ERG	PhysioNet (Goldberger et al., 2000)	Standardised clinical electrophysiology recordings

The labelling gap. None of the resources in Table 3.3 ships Healthy / Glaucoma / Optic Neuritis-CVI diagnoses. Real training therefore requires a labelled, paired clinical cohort or a clinical partner able to label a subset. The data interface described in Section 3.6 is ready for such a cohort: a manifest with the columns `subject_id, mri_path, vep_path, fs, label` is all that is needed.

3.5 SYNTHETIC DATA GENERATION

To close the labelling gap for software development, `src/data/synthetic.py` generates class-conditioned MRI-like volumes and VEP-like waveforms. The conditioning is deliberately simple but structured so that a model can learn from it. The data are useful for software validation and for testing the learning loop; they are not evidence of clinical performance.

3.5.1 SYNTHETIC MRI VOLUMES

A volume of shape (80, 96, 64) is drawn from a normal distribution with mean 100 and standard deviation 15, clipped at zero. Class conditioning is applied to one octant-like region: for **Glaucoma** the intensities in the block with indices [D/2:, :H/2, :W/2] are multiplied by 0.4 (a proxy for atrophy), and for **Optic Neuritis / CVI** the block [:D/2, H/2:, W/2:] is multiplied by 1.8 (a proxy for a hyperintense lesion). Healthy volumes are left unstructured. Figure 3.2 shows example axial slices.

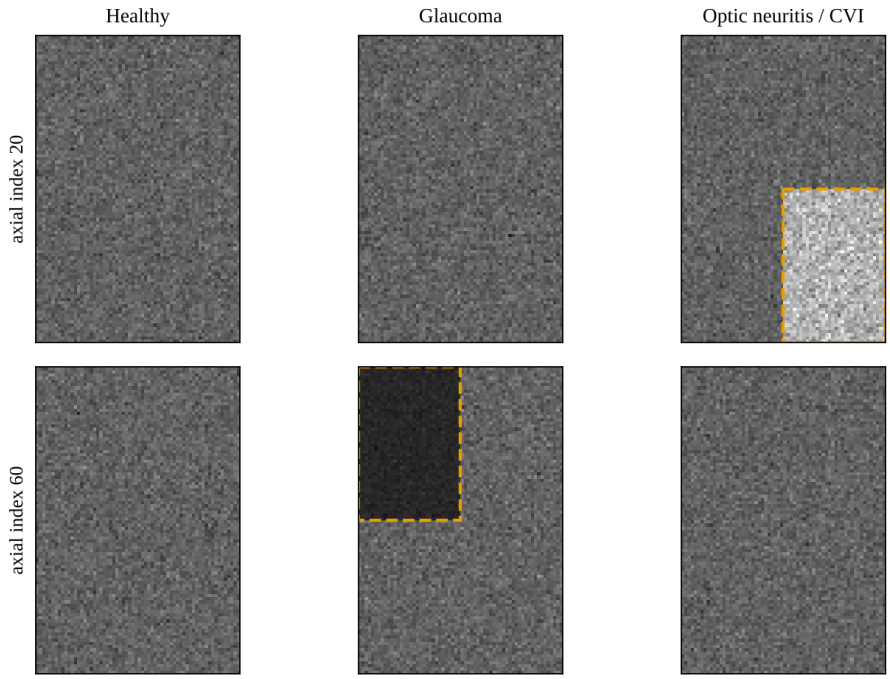


Figure 3.2: Axial slices of synthetic MRI volumes generated by the project's own generator (seeded). Dashed boxes mark the class-specific region: lesion proxy (Optic neuritis / CVI, index 20) and atrophy proxy (Glaucoma, index 60).

3.5.2 SYNTHETIC VEP WAVEFORMS

A waveform sampled at 250 Hz for 2 s (500 samples) is built from three Gaussian components that mimic the N75-P100-N145 complex, with latency τ and amplitude A that depend on the class (Table 3.4):

$$s(t) = A\, e^{-\frac{(t-\tau)^2}{2\sigma_1^2}} - 0.5A\, e^{-\frac{(t-\tau+0.025)^2}{2\sigma_2^2}} - 0.4A\, e^{-\frac{(t-\tau-0.045)^2}{2\sigma_3^2}} \tag{3.1}$$

with σ_1 = 12 ms, σ_2 = 10 ms and σ_3 = 15 ms. Three artefacts are then added: a slow baseline drift 0.3 sin(2π · 0.3 t), mains interference 0.15 sin(2π · 50 t) and Gaussian muscle noise with standard deviation 0.3. These are exactly the artefacts that the conditioning pipeline of Section 3.8 is designed to remove.

Table 3.4: Class-conditioned parameters of the synthetic generators.

Class (label)	MRI conditioning	P100 latency τ	P100 amplitude A
Healthy Control (0)	None (background-like)	100 ms	6.0
Glaucoma (1)	Block × 0.4 (atrophy proxy)	108 ms	2.5
Optic Neuritis / CVI (2)	Block × 1.8 (lesion proxy)	140 ms	4.0

3.5.3 DATASET BEHAVIOUR

`SyntheticNeuroDataset` creates a balanced list of labels and shuffles it deterministically from a seed. The raw sample for an index is generated when the item is requested, using a shared random-number generator. Consequently the same index is not guaranteed to return identical noise on repeated access. The function `write_synthetic_dataset` writes the samples to disk as `sub-XXXX_mri.nii.gz` and `sub-XXXX_vep.csv` files with a `manifest.csv`, which is how the end-to-end tests exercise the real file path.

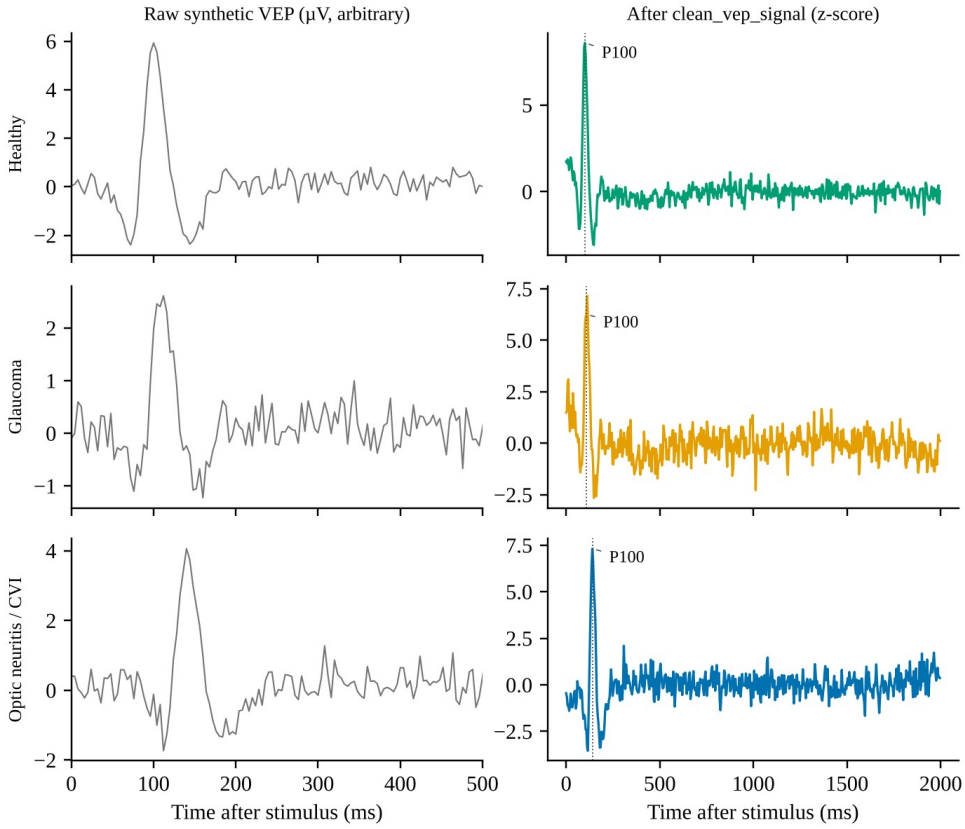


Figure 3.3: Synthetic VEP waveforms for the three classes, before (left) and after (right) the project's conditioning function `clean_vep_signal`. Dotted line: ground-truth P100 latency.

3.6 DATA INTERFACE

`PairedNeuroDataset` reads a manifest CSV with the columns `subject_id`, `mri_path`, `vep_path`, `fs`, `label` and raises an error if the manifest is empty. For each row, `__getitem__` performs the following steps: (1) load and standardise the MRI with `preprocess_mri_file`; (2) read the VEP CSV with `numpy.loadtxt`, skipping the header; (3) take the sampling rate from the row, falling back to the configured 250 Hz when it is missing; (4) condition the waveform

with `clean_vep_signal`; and (5) return a dictionary with the MRI tensor (1, D, H, W), the VEP tensor (1, L), a long-integer label and the subject identifier.

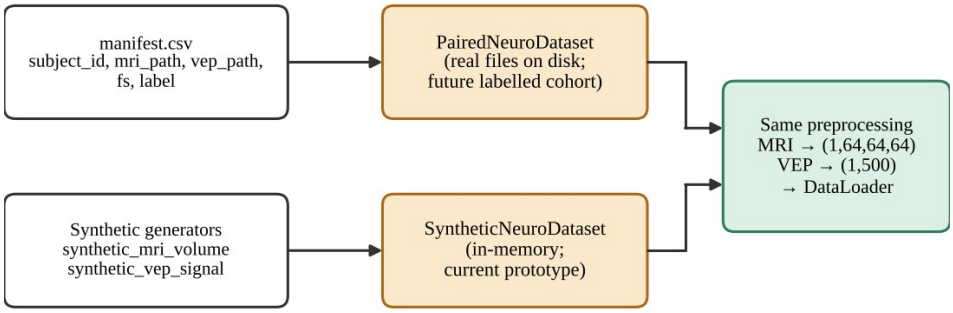


Figure 3.4: The two dataset classes share one preprocessing path. The manifest-driven class is intended for a future labelled cohort; the in-memory synthetic class is used in the current prototype.

3.7 MRI PREPROCESSING

MRI standardisation (`src/preprocessing/mri_transforms.py`) consists of five steps (Figure 3.5). The parameters are held in `MRIConfig`.

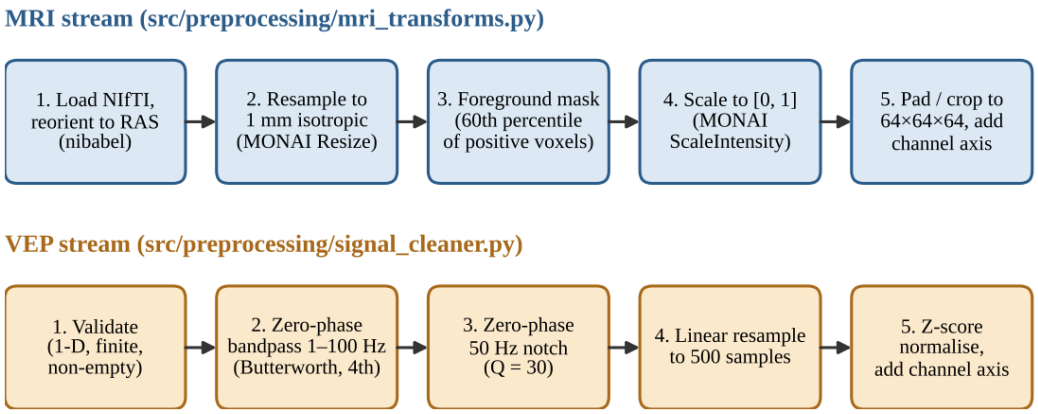


Figure 3.5: Preprocessing chains for the MRI stream (top) and the VEP stream (bottom), with the function responsible for each step.

1. **Load and canonicalise.** `load_nifti` reads the NIfTI file with NiBabel and reorients it to the closest canonical (RAS) frame. It verifies that the image is three-dimensional and that all voxel spacings are positive, and returns the data, the affine and the voxel zooms.
2. **Resample to isotropic spacing.** For voxel zooms z and target spacing s (1 mm), the new shape is computed from the original shape n (Equation 3.2) and the volume is resampled with MONAI's trilinear `Resize`.

3. **Foreground masking.** The *p*-th percentile (default 60) of the strictly positive voxels is computed and all voxels below it are set to zero (Equation 3.3).
4. **Intensity scaling.** If the masked volume has a non-zero range, MONAI `ScaleIntensity` maps it to [0, 1] (Equation 3.4); a constant volume is preserved as a float array.
5. **Pad or crop.** `ResizeWithPadOrCrop` brings the volume to (64, 64, 64) and the channel axis is added, giving (1, 64, 64, 64).

$$n_i' = \text{round}\left(\frac{n_i z_i}{s_i}\right) \tag{3.2}$$

$$\tau_p = P_p(\{x : x > 0\}), \qquad \tilde{x} = x \ \text{ if } x \geq \tau_p, \quad \tilde{x} = 0 \ \text{ otherwise} \tag{3.3}$$

$$x' = \frac{\tilde{x} - \min(\tilde{x})}{\max(\tilde{x}) - \min(\tilde{x})} \tag{3.4}$$

Important caveat. The percentile mask is a lightweight, dependency-free stand-in for skull-stripping. It is *not* clinical-grade brain extraction and does not replace validated tools such as HD-BET (Isensee et al., 2019) or SynthStrip (Hoopes et al., 2022). It is isolated in the function `foreground_mask_threshold` so that it can be replaced without changing any downstream code.

Table 3.5: MRI preprocessing parameters (`MRIConfig`).

Parameter	Default	Meaning
target_shape	(64, 64, 64)	Output spatial size after pad or crop
target_spacing	(1.0, 1.0, 1.0) mm	Isotropic resampling target
orientation	RAS	Canonical orientation
foreground_percentile	60.0	Percentile of positive voxels used as the mask threshold

3.8 VEP PREPROCESSING

VEP conditioning (`src/preprocessing/signal_cleaner.py`) removes baseline drift, high-frequency muscle noise and mains interference without shifting the waveform in time. The parameters are held in `VEPConfig` (Table 3.6).

1. **Validate.** The input is converted to 64-bit floats and rejected if it is not one-dimensional, is empty, or contains NaN or infinite values.
2. **Band-pass.** `butterworth_bandpass` applies a fourth-order Butterworth band-pass between 1 Hz and 100 Hz in second-order-section form with

`sosfiltfilt`, i.e. forward and backward. The upper edge must lie below the Nyquist frequency (125 Hz at 250 Hz sampling) and the record must be long enough for zero-phase padding; otherwise an error is raised.

3. **Notch.** `notch_filter` applies a zero-phase IIR notch at 50 Hz with quality factor 30 to suppress AC power-line interference.
4. **Resample.** `resample_to_length` linearly interpolates the record to exactly 500 samples.
5. **Normalise.** `zscore_normalize` subtracts the mean and divides by the standard deviation plus a small constant (Equation 3.6); the result is returned as 32-bit floats and a channel axis is added.

$$|H(j\omega)|^2 = \frac{1}{1 + (\omega/\omega_c)^{2n}} \quad (n = 4), \qquad H_{\mathrm{zp}}(\omega) = |H(j\omega)|^2 \tag{3.5}$$

$$\hat{x}_k = \frac{x_k - \mu}{\sigma + \varepsilon}, \qquad \varepsilon = 10^{-8} \tag{3.6}$$

Why zero-phase filtering. An ordinary single-pass (causal) filter delays different frequency components by different amounts (its group delay). Because the P100 latency is itself the clinical measurement, such a delay would corrupt the very quantity of interest. Filtering forward and then backward cancels the phase response, so the effective frequency response is the squared magnitude of the single-pass filter and has zero phase (Oppenheim & Schafer, 2010; Gustafsson, 1996). The price is that the filter is non-causal and the effective roll-off is steeper; both are acceptable for offline analysis.

Table 3.6: VEP preprocessing parameters (`VEPConfig`).

Parameter	Default	Meaning
length	500	Samples after resampling
sample_rate	250 Hz	500 samples span a 2.0 s sweep
bandpass_low / bandpass_high	1.0 Hz / 100.0 Hz	Edges of the fourth-order Butterworth band-pass
notch_freq	50 Hz	Mains frequency
notch_quality	30	Q factor; notch bandwidth = f_0 / Q ≈ 1.67 Hz

3.9 EVALUATION STRATEGY

Because the data are synthetic, the evaluation is organised around evidence that is meaningful for a software prototype. It is designed to answer the research questions of Section 1.4 without implying clinical performance.

Table 3.7: Evaluation strategy: what is measured, how, and which objective it supports.

Evidence	Method	Supports
Signal-conditioning behaviour	The project's own functions are run on 150 synthetic VEPs (50 per class, fixed seeds): frequency response, mains attenuation, group delay, detected P100 latency against ground truth	O2, RQ2, H1
Synthetic data characterisation	Region-of-interest statistics of generated MRI volumes; effect of the foreground mask; per-class VEP statistics	O2, O7
Model size and cost	Trainable parameters and multiply-accumulate operations counted analytically from the layer definitions	O3, O4, RQ3
Software verification	86 test functions (unit and integration); statement coverage from the recorded coverage data	O1, O6, H3
Training acceptance criteria	Thresholds asserted by the training-loop integration tests (loss reduction, above-chance validation accuracy, checkpoint round-trip)	O4, RQ3, H2

Classification metrics such as accuracy, sensitivity, specificity or area under the curve are *not* reported as evidence of diagnostic ability, because they would be computed on a synthetic task that was designed to be separable. The reasons are discussed further in Chapter 6.

Chapter 4
System Design and Implementation

4.1 ARCHITECTURE OVERVIEW

The Cross-Attention Fusion Network (CAFN), implemented in `src/models/cafn.py`, has three parts: two modality-specific encoders that turn each input into a sequence of 128-dimensional tokens, a cross-attention fusion module that lets spatial tokens query temporal tokens, and three linear classification heads (fused, spatial-auxiliary and temporal-auxiliary). Figure 4.1 shows the layer-by-layer structure with the tensor shape after each operation for a batch of size *B*.

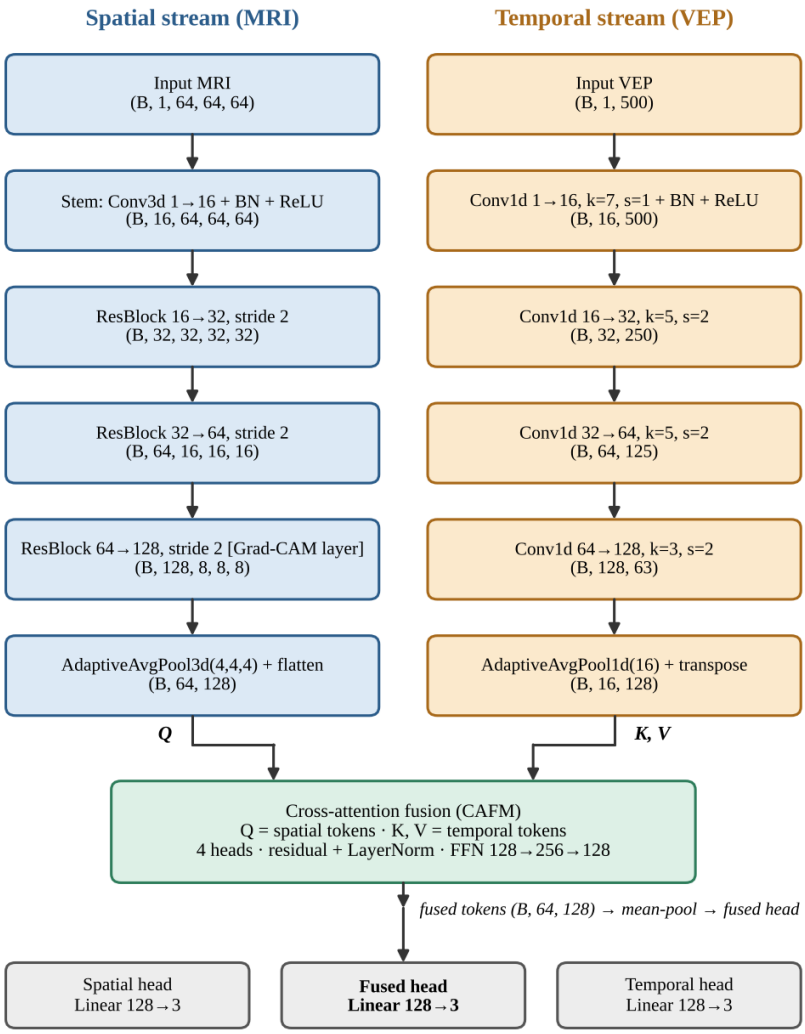


Figure 4.1: CAFN architecture with tensor shapes (default configuration). The two auxiliary heads read the mean-pooled unimodal tokens; the fused head reads the mean-pooled fused tokens.

The complete model has 1,055,081 trainable parameters, about 4.2 MB in 32-bit floating point (Section 5.4).

4.2 SPATIAL ENCODER (3D RESIDUAL CNN)

`SpatialEncoder3D` (`src/models/spatial_encoder.py`) converts a standardised MRI volume of shape (B, 1, 64, 64, 64) into a sequence of 64 tokens. It consists of a convolutional stem, three residual blocks that each halve the spatial resolution and increase the channel count, and an adaptive average pool to a 4 × 4 × 4 grid (Table 4.1).

Table 4.1: Spatial encoder layers (batch size 1) and trainable parameters.

Layer	Operation	Output shape	Parameters
Stem	Conv3d 1→16 (3×3×3, stride 1) + BatchNorm + ReLU	(16, 64, 64, 64)	464
Residual block 1	16→32, stride 2	(32, 32, 32, 32)	42,176
Residual block 2	32→64, stride 2	(64, 16, 16, 16)	168,320
Residual block 3	64→128, stride 2 (Grad-CAM target `last_conv`)	(128, 8, 8, 8)	672,512
Pool and tokens	AdaptiveAvgPool3d (4,4,4); flatten; transpose	(64, 128)	0
Total			**883,472**

Each residual block applies two 3 × 3 × 3 convolutions with batch normalisation (Ioffe & Szegedy, 2015); the first has the block's stride. The input is added to the output through an identity shortcut, or through a 1 × 1 × 1 convolution with batch normalisation when the stride or the channel count changes, and a ReLU follows the addition (He et al., 2016). The spatial size of a convolutional layer with kernel k, stride s and padding p follows Equation 4.1.

$$L_{\text{out}} = \left\lfloor \frac{L_{\text{in}} + 2p - k}{s} \right\rfloor + 1 \tag{4.1}$$

After the three stride-2 blocks the feature map is 8 × 8 × 8; adaptive average pooling reduces it to 4 × 4 × 4, so each of the 64 tokens summarises a 2 × 2 × 2 neighbourhood of feature-map positions, i.e. a nominal 16 × 16 × 16 voxel cube of the standardised (1 mm) volume. The final residual block is stored under the name `last_conv` so that the Grad-CAM implementation can attach its hooks to it.

4.3 TEMPORAL ENCODER (1D CNN)

`TemporalEncoder1D` (`src/models/temporal_encoder.py`) maps a conditioned VEP of shape (B, 1, 500) to 16 tokens through four convolutional stages,

each followed by batch normalisation and a ReLU, and an adaptive average pool (Table 4.2).

Table 4.2: Temporal encoder layers (batch size 1) and trainable parameters.

Stage	Convolution	Output length	Parameters
1	Conv1d 1 → 16, kernel 7, stride 1	500	144
2	Conv1d 16 → 32, kernel 5, stride 2	250	2,624
3	Conv1d 32 → 64, kernel 5, stride 2	125	10,368
4	Conv1d 64 → 128, kernel 3, stride 2	63	24,832
Pool	AdaptiveAvgPool1d (16); transpose → (16, 128)	16	0
Total			**37,968**

The cascade has a receptive field of 27 samples (108 ms at 250 Hz) and an output stride of 8 samples (32 ms). The 63 output positions are averaged into 16 tokens, so each token nominally summarises about 125 ms of the 2 s epoch (Figure 4.2). The tokens are *learned feature positions*: without further temporal registration they should not be read as calibrated millisecond bins.

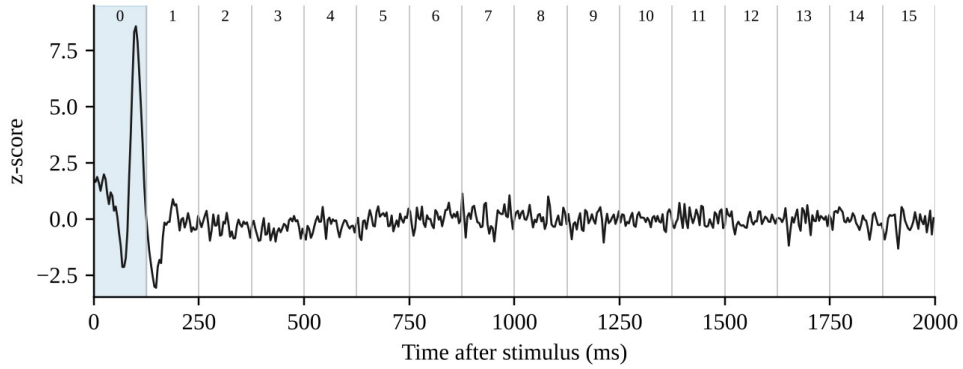


Figure 4.2: Nominal mapping of the 16 temporal tokens onto the 2 s epoch (125 ms per token) over an example conditioned VEP. The shaded region is token 0.

4.4 CROSS-ATTENTION FUSION MODULE

The fusion module (`src/models/cross_attention.py`) is the central component. The spatial tokens X_s (64 × 128) supply the queries; the temporal tokens X_t (16 × 128) supply the keys and values (Equation 4.2). Each spatial token can therefore attend to any of the 16 temporal tokens.

$$Q = X_s W_Q, \qquad K = X_t W_K, \qquad V = X_t W_V \tag{4.2}$$

With d_{model} = 128 and H = 4 heads, each head has dimension d_h = 32. For head h the scaled dot-product attention is computed (Vaswani et al., 2017), with a softmax over the 16 temporal tokens, followed by dropout on the attention weights (p = 0.1):

$$A_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_h}}\right), \qquad O_h = A_h V_h \tag{4.3}$$

The head outputs are concatenated and projected, added to the original spatial tokens through a residual connection, normalised (Ba et al., 2016), passed through a position-wise feed-forward block (128 → 256 → 128 with ReLU and dropout) and normalised again after a second residual connection:

$$Z = \text{LN}(X_s + \text{Concat}(O_1, \ldots, O_H)\, W_O), \qquad F = \text{LN}(Z + \text{FFN}(Z)) \tag{4.4}$$

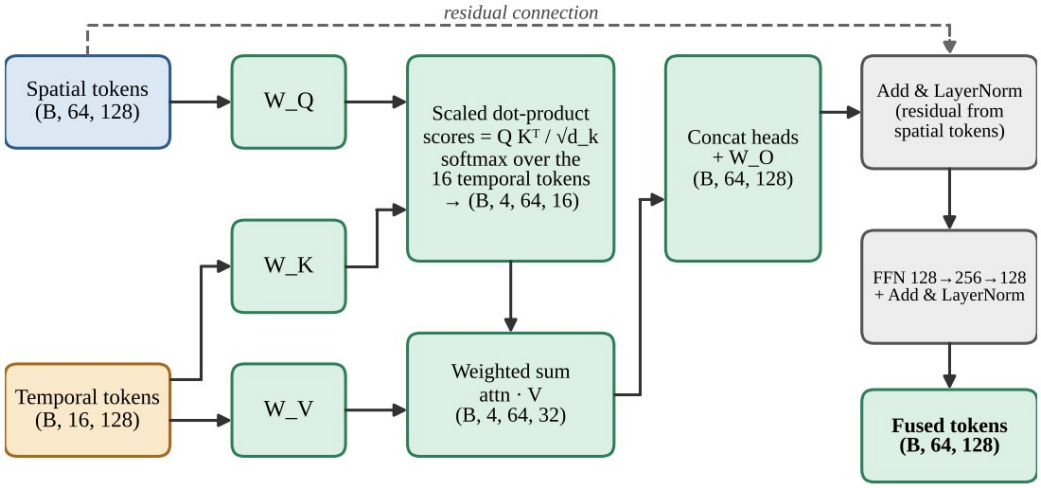


Figure 4.3: Structure of the cross-attention fusion module (CAFM). The attention-weight tensor has shape (B, 4, 64, 16) and is returned for inspection.

The output F keeps the shape (B, 64, 128): the spatial-token sequence is preserved and each token is enriched with temporal context. The module accepts queries and keys of different lengths and rejects configurations in which d_{model} is not divisible by the number of heads. The projection matrices, the output projection and the feed-forward block account for 132,480 trainable parameters (Table 5.5).

4.5 CLASSIFICATION HEADS AND MULTI-TASK LOSS

`CAFN.forward` mean-pools three token sequences, the fused tokens, the spatial tokens and the temporal tokens, and passes each pooled 128-dimensional embedding through its own linear layer to obtain three sets of logits of shape (B, 3). The fused head is the primary prediction used at evaluation time. The two auxiliary heads provide unimodal supervision: they encourage each encoder to carry class information on its own and reduce the chance that the fused pathway learns to ignore one modality

entirely. With cross-entropy L_f, L_s and L_t for the fused, spatial and temporal heads, the objective is

$$L_{\text{total}} = L_f + \lambda\,(L_s + L_t), \qquad \lambda = 0.3 \tag{4.5}$$

The function `CAFN.multi_task_loss` returns the total together with the three component values, so that each can be logged.

4.6 TRAINING PROCEDURE

Training is implemented in `src/train.py` and summarised in Figure 4.4. For each epoch, `train_model` performs the following steps.

1. Put the model in training mode and iterate over a shuffled `DataLoader` (batch size 8).
2. Move the MRI, VEP and label tensors to the selected device (default: CPU).
3. Clear gradients, run the forward pass and compute the multi-task loss with the configured auxiliary weight.
4. Back-propagate and take an Adam step (learning rate 10^{-3}, weight decay 10^{-4}); accumulate the sample-weighted training loss.
5. If a validation loader is supplied, evaluate the fused-head loss and accuracy with gradients disabled and the model in evaluation mode.

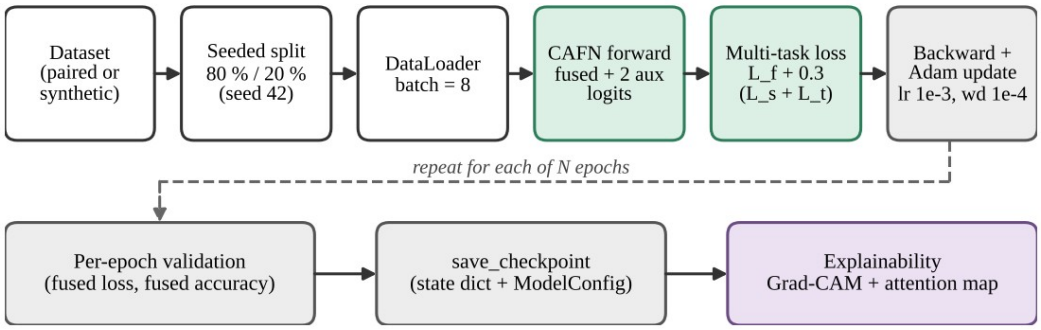


Figure 4.4: Training and evaluation flow. A seeded permutation splits the data; each epoch is followed by validation; the trained model can be checkpointed and passed to the explainability tools.

Table 4.3: Training configuration (`TrainConfig`) and model configuration (`ModelConfig`) at the default values.

Parameter	Default	Parameter	Default
batch_size	8	d_model	128
epochs	20	num_heads	4
lr (Adam)	1e-3	spatial_tokens / spatial_pool	64 / (4, 4, 4)
weight_decay	1e-4	temporal_tokens	16
aux_loss_weight (λ)	0.3	ffn_hidden	256
val_split	0.2	dropout	0.1
seed	42	num_classes	3
checkpoint_dir	checkpoints		

Splitting. `train_val_split` draws a deterministic random permutation with a seeded generator, assigns the first `max(1, int(n · val_split))` indices to validation and the rest to training, and falls back to using all samples for training when a very small dataset would otherwise leave none. **Checkpoints.** `save_checkpoint` stores the model's state dictionary together with its `ModelConfig`; `load_checkpoint` rebuilds the model from the stored configuration and loads the weights on the CPU. **Command line.** The module can be run as `python -m src.train --epochs 20 --synthetic-samples 64`; this entry point trains on the synthetic dataset only. **Not implemented.** The loop has no learning-rate scheduler, early stopping, mixed precision, class weighting or per-epoch checkpointing.

4.7 EXPLAINABILITY

4.7.1 3D GRAD-CAM ON THE FUSED LOGIT

`GradCAM3D` (`src/explainability/grad_cam_3d.py`) registers a forward hook and a backward hook on the spatial encoder's last residual block. It back-propagates from the selected *fused* logit, by default that of the predicted class, so that the gradients flow through the cross-attention module into the spatial encoder and the map reflects the multimodal decision. With activations A^k of channel k and class score y^c, the channel weights are the spatially averaged gradients and the map is a rectified weighted sum (Selvaraju et al., 2017):

$$\alpha_k = \frac{1}{Z}\sum_{i,j,l}\frac{\partial y^c}{\partial A^k_{ijl}}, \qquad M = \mathrm{ReLU}\left(\sum_k \alpha_k A^k\right) \tag{4.6}$$

The map is upsampled by trilinear interpolation to the input shape and normalised per sample to [0, 1]. The helper `plot_gradcam_slices` overlays it on the mid axial, coronal and sagittal slices and saves a PNG. The model is returned to its original training or evaluation mode and the hooks are removed after use.

4.7.2 CROSS-ATTENTION HEATMAP

`cross_attention_heatmap` averages the attention tensor (B, H, S, T) over the batch and the heads to a matrix of shape (S, T), 64 × 16 at the default configuration, and `plot_cross_attention` renders it as a heatmap with spatial tokens on one axis and temporal tokens on the other.

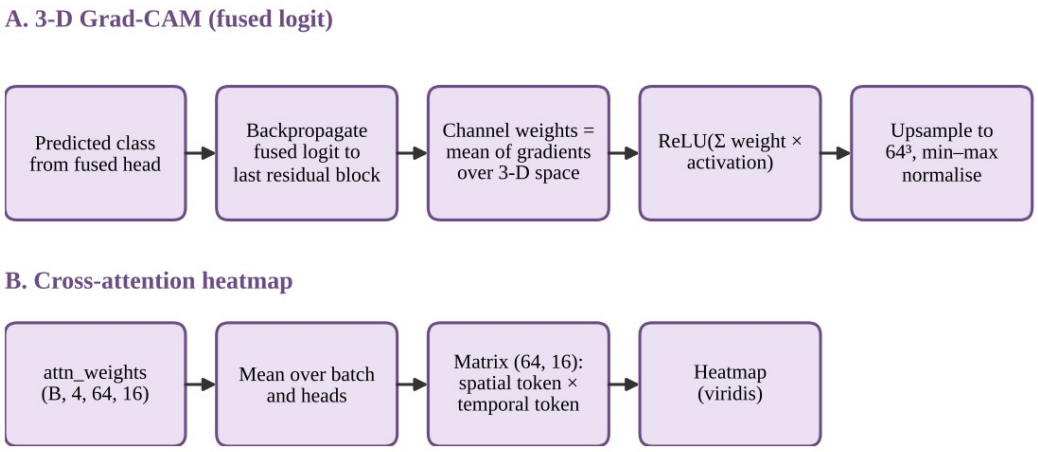


Figure 4.5: Generation of the two explanation artefacts: the Grad-CAM map (A) and the cross-attention heatmap (B).

Interpretation caveat. Both artefacts are inspection aids. The Grad-CAM map shows which feature locations influenced the selected logit; it does not by itself identify V1, V2, V3, the optic nerve or any other anatomical structure. The heatmap axes are learned token indices, not registered brain regions or calibrated milliseconds. Chapter 6 discusses why attention and saliency maps must be validated before being offered as clinical evidence.

4.8 SOFTWARE STRUCTURE AND CONFIGURATION

The code base is organised by responsibility (Figure 4.6 and Table 4.4). The configuration module is the only place in which shapes and hyper-parameters are defined; the remaining modules import it.

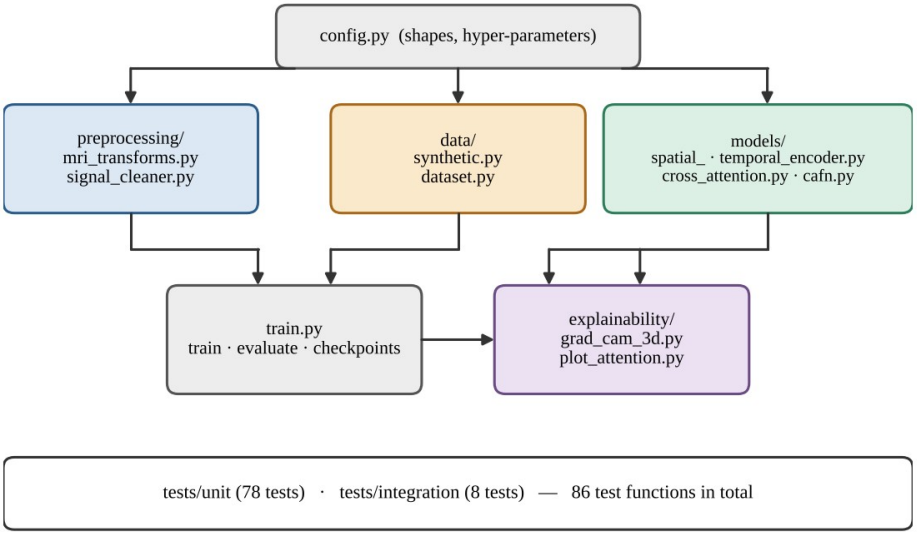


Figure 4.6: Module structure and dependencies of the code base.

Table 4.4: Source modules, line counts and responsibilities.

Module	Lines	Responsibility
src/config.py	64	Frozen dataclasses (MRI, VEP, model, training); class names
src/data/synthetic.py	125	Synthetic MRI and VEP generators; on-disk dataset writer
src/data/dataset.py	88	`PairedNeuroDataset` (manifest) and `SyntheticNeuroDataset`
src/preprocessing/mri_transforms.py	101	NIfTI loading, resampling, masking, scaling, pad/crop
src/preprocessing/signal_cleaner.py	109	Validation, band-pass, notch, resampling, z-score
src/models/spatial_encoder.py	61	3D residual CNN producing spatial tokens
src/models/temporal_encoder.py	35	1D CNN producing temporal tokens
src/models/cross_attention.py	75	Multi-head cross-attention and fusion block
src/models/cafn.py	61	Full model, heads and multi-task loss
src/train.py	161	Training, evaluation, split, checkpoints, command line
src/explainability/grad_cam_3d.py	127	3D Grad-CAM engine and slice plots
src/explainability/plot_attention.py	47	Attention reduction and heatmap

4.9 TEST DESIGN

The test suite (`tests/`) contains 86 test functions in 12 files: 78 unit tests and 8 integration tests. A shared `tiny_config` fixture scales the model down (MRI shape 32^3, VEP length 200, $d_{\text{model}} = 32$, two heads, eight spatial and eight temporal tokens, three epochs) so that tests run quickly while exercising the same interfaces. Fixtures also provide a seeded random generator, a factory that writes synthetic NIfTI files

with non-isotropic spacing, and a VEP-like array with known 10 Hz and 50 Hz components. All tests run with fixed seeds.

Table 4.5: Test files and what they verify.

Test file	Tests	Properties verified
unit/test_signal_cleaner.py	18	Output shape; attenuation of drift and mains hum; preservation of in-band content; rejection of invalid input (non-1D, NaN, too short, cut-off above Nyquist); mean 0 / std 1; P100-like peak latency is not shifted
unit/test_mri_transforms.py	12	RAS orientation; voxel zooms; low-intensity voxels zeroed; all-zero volume unchanged; anisotropic spacing resampled along the correct axis; output shape and range; constant volumes produce no NaN; full file pipeline
unit/test_cross_attention.py	8	Output and attention shapes; attention rows sum to one; mismatched query/key lengths; divisibility check; finite outputs; gradients reach all parameters; temporal input actually changes the output
unit/test_grad_cam_3d.py	8	Map shape and [0, 1] range; explicit target class respected; hooks removed; training mode restored; PNG written; mismatched shapes rejected
unit/test_cafn_model.py	7	Output keys and shapes; batch size one; finite logits; loss equals the weighted sum of parts; backward reaches every parameter; optimiser step changes weights
unit/test_dataset.py	7	Item shapes and types; length; balanced labels; data-loader batching; seed dependence; manifest reading; empty-manifest error
unit/test_spatial_encoder.py	5	Output shape; batch size one; gradient flow; hook target participates in forward pass; eval-mode determinism
unit/test_plot_attention.py	5	Reduction over batch and heads; non-negative values; dimensionality check; PNG written; matches manual reduction
unit/test_temporal_encoder.py	4	Output shape; batch size one; gradient flow; robustness to input length
unit/test_train_utils.py	4	Split partitions the dataset; single-sample fallback; determinism for a fixed seed; empty loader returns NaN
integration/ test_pipeline_end_to_end.py	4	Raw files → preprocessing → model → loss → backward → optimiser step; shape compatibility; Grad-CAM and attention plots from real pipeline output; glaucoma and neuritis VEP peaks distinguishable
integration/test_training_loop.py	4	Training loss falls substantially; model separates synthetic classes above a minimum validation accuracy; training without a validation set; checkpoint round-trip leaves logits unchanged

Chapter 5
Results

5.1 INTRODUCTION

This chapter presents the measurements obtained for the project. Following the evaluation strategy of Section 3.9, the results concern (i) the behaviour of the signal-conditioning pipeline, (ii) the properties of the synthetic data, (iii) the size and computational cost of the model, and (iv) the results of the automated verification (test suite and coverage). They are presented without interpretation; the interpretation follows in Chapter 6.

Two points about what is *not* reported are stated here to avoid ambiguity. First, no diagnostic-performance metrics are reported, because no labelled clinical data exist and the synthetic task is separable by construction. Second, measured training curves and validation accuracies are not reported, because the training experiments could not be executed in the environment used to prepare this report; the training behaviour is instead described through the acceptance criteria that the repository's integration tests enforce (Section 5.7). All measurements below were produced by running the project's own functions on seeded synthetic data (50 trials per class, 150 in total; sampling rate 250 Hz; 500 samples per record) or by analysing the repository's code and recorded test data.

5.2 SIGNAL CONDITIONING RESULTS

5.2.1 FREQUENCY RESPONSE

The combined frequency response of the zero-phase conditioning filters (fourth-order Butterworth band-pass, 1–100 Hz, followed by a 50 Hz notch with Q = 30, each applied forward and backward) is shown in Figure 5.1 and sampled at selected frequencies in Table 5.1. The −6 dB bandwidth of the notch (the −3 dB point of a single pass) extends from 49.18 to 50.83 Hz.

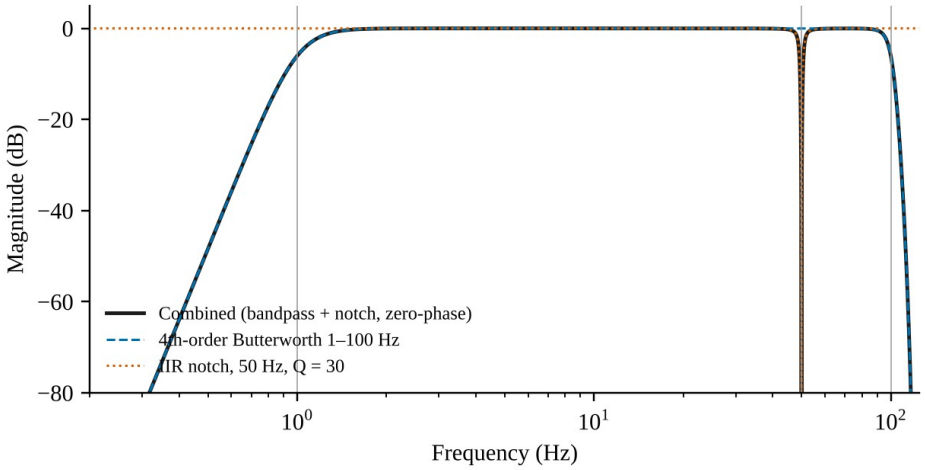


Figure 5.1: Magnitude response of the conditioning filters in the effective (forward-backward) form.

Table 5.1: Effective magnitude response (dB) at selected frequencies.

Frequency (Hz)	Band-pass	Notch	Combined
0.3	-82.73	0.00	-82.73
1.0	-5.78	0.00	-5.78
10.0	0.00	0.00	0.00
49.0	0.00	-4.52	-4.52
50.0	0.00	-97.45	-97.45
51.0	0.00	-4.65	-4.65
100.0	-6.03	0.00	-6.03

5.2.2 MAINS-INTERFERENCE ATTENUATION

A least-squares sinusoid fit at 50 Hz was used to measure the amplitude of the injected mains component before and after band-pass and notch filtering. Averaged over the 150 trials, the fitted 50 Hz amplitude fell from 0.152 to 0.0125 (arbitrary units), a mean reduction of 22.0 dB. For reference, the residual amplitude fitted at 47 Hz after filtering, a frequency at which no component was injected, was 0.0212. Figure 5.2 shows the amplitude spectrum of one example record.

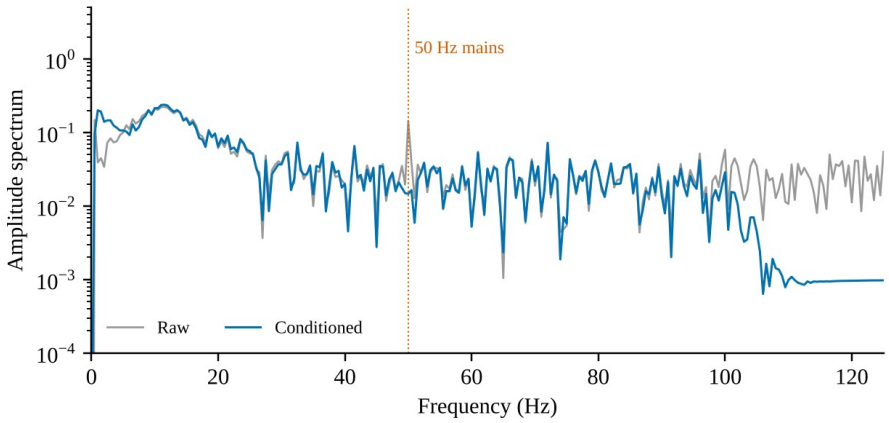


Figure 5.2: Amplitude spectrum of one synthetic VEP before and after conditioning. The dotted line marks 50 Hz.

5.2.3 P100 LATENCY

The P100 latency was estimated as the time of the maximum within 60–220 ms. Table 5.2 compares the estimates from the raw signal, from the project's zero-phase conditioning, and from an ordinary single-pass (causal) implementation of the same filters, with the ground-truth latency of the generator. Estimates are quantised by the sampling interval of 4 ms.

Table 5.2: Detected P100 latency (ms; mean ± SD over 50 trials per class) and mean error against the ground truth.

Class	True	Raw	Zero-phase	Single-pass	Err. zero-phase	Err. single-pass
Healthy Control	100	101.8 ± 2.3	100.6 ± 1.9	101.1 ± 1.8	+0.64	+1.12
Glaucoma	108	107.4 ± 2.7	108.0 ± 2.9	108.2 ± 2.3	+0.00	+0.24
Optic Neuritis / CVI	140	141.3 ± 2.5	140.8 ± 2.1	141.0 ± 2.7	+0.80	+1.04

With zero-phase conditioning, 149 of 150 trials (99.3%) gave a latency within one sample period (4 ms) of the ground truth; the largest absolute error was 8 ms, which occurred in one Glaucoma trial. For the single-pass implementation the corresponding count was 149 of 150, and the largest raw-signal error was 8 ms. The group delay of the single-pass filter, computed from its transfer function, is given in Table 5.3 (the zero-phase pipeline has zero group delay at all frequencies) and plotted in Figure 5.3.

Table 5.3: Group delay of the single-pass (causal) implementation of the same filters.

Frequency (Hz)	Group delay (ms)
1	593.1
2	120.7
5	18.8
10	6.1
20	3.1
30	2.8
40	3.9
60	4.8

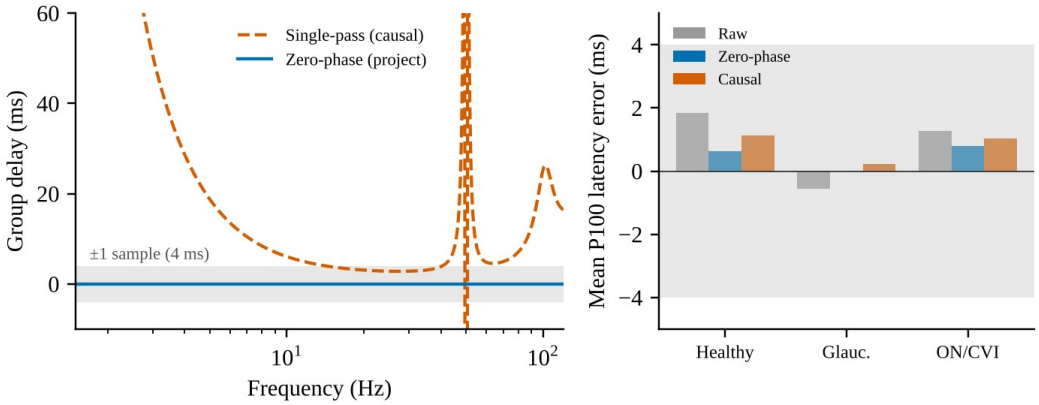


Figure 5.3: Left: group delay of the single-pass filters versus the zero-phase pipeline (shaded band: ±1 sample, 4 ms). Right: mean P100 latency error by class (shaded band: ±4 ms).

5.2.4 CONDITIONED-WAVEFORM STATISTICS

After conditioning and z-scoring, the peak value of the P100 window was 8.38 ± 0.27 for Healthy Control, 6.85 ± 0.41 for Glaucoma and 7.51 ± 0.54 for Optic Neuritis / CVI (mean ± SD, 50 trials per class). Figure 5.4 shows the distributions of the detected latency and of the peak value.

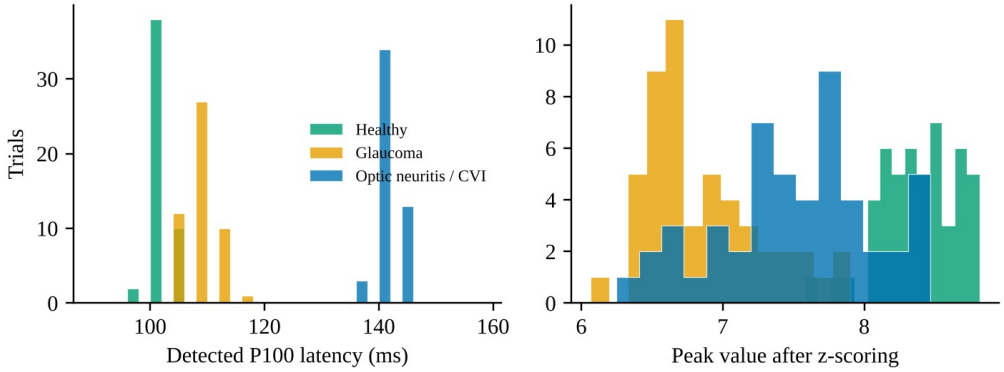


Figure 5.4: Distributions over 50 synthetic trials per class: detected P100 latency after conditioning (left) and peak value after z-scoring (right).

5.2.5 NOMINAL TEMPORAL-TOKEN BINS

The temporal encoder reduces the 500-sample record to 16 tokens, so that each token nominally spans 125 ms of the 2.0 s record. The ground-truth P100 latencies of the three classes (100, 108 and 140 ms) fall in token 0 for Healthy Control, token 0 for Glaucoma and token 1 for Optic Neuritis / CVI (the token mapping is illustrated in Figure 4.2).

5.3 SYNTHETIC MRI CHARACTERISATION

One seeded synthetic volume of shape (80, 96, 64) was generated per class and the intensity statistics of the class-specific regions of interest (ROI) were measured, together with the effect of the project's foreground-mask function at its default 60th percentile (Table 5.4). The ROIs are the atrophy-proxy block [D/2:, :H/2, :W/2] and the lesion-proxy block [:D/2, H/2:, W/2:] described in Section 3.5.

Table 5.4: Synthetic MRI: mean intensity of regions of interest and effect of the 60th-percentile foreground mask.

Class	Mean, rest of volume	Mean, atrophy ROI	Mean, lesion ROI	Voxels zeroed (%)	Atrophy ROI retained (%)	Lesion ROI retained (%)
Healthy Control	100.0	99.9	100.0	60.0	39.7	40.0
Glaucoma	100.0	40.0	100.0	60.0	0.0	45.7
Optic Neuritis / CVI	100.0	100.1	179.9	60.0	31.7	99.7

5.4 MODEL COMPLEXITY

The number of trainable parameters and the multiply-accumulate (MAC) operations per sample were counted analytically from the layer definitions in `src/models/*.py` at the default configuration (Table 5.5). Batch-normalisation running statistics are buffers, not trainable parameters, and are excluded; MAC counts include convolutions and linear layers and the attention matrix products, and exclude normalisation, activation, pooling and the softmax. In total the model has 1,055,081 trainable parameters and about 2.53 billion MACs per sample.

Table 5.5: Trainable parameters and multiply-accumulates (millions) per sample at the default configuration.

Component	Parameters	MACs (millions)
Spatial stem (Conv3d + BN)	464	113.25
Spatial residual block 1 (16 → 32)	42,176	1,375.73
Spatial residual block 2 (32 → 64)	168,320	687.87

Component	Parameters	MACs (millions)
Spatial residual block 3 (64 → 128)	672,512	343.93
Temporal encoder (4 stages)	37,968	3.52
Cross-attention projections (W_Q, W_K, W_V, W_O)	66,048	2.88
Feed-forward block (128 → 256 → 128)	65,920	4.19
Layer normalisation (×2)	512	—
Classification heads (×3)	1,161	0.00
Total	**1,055,081**	**2,531.38**

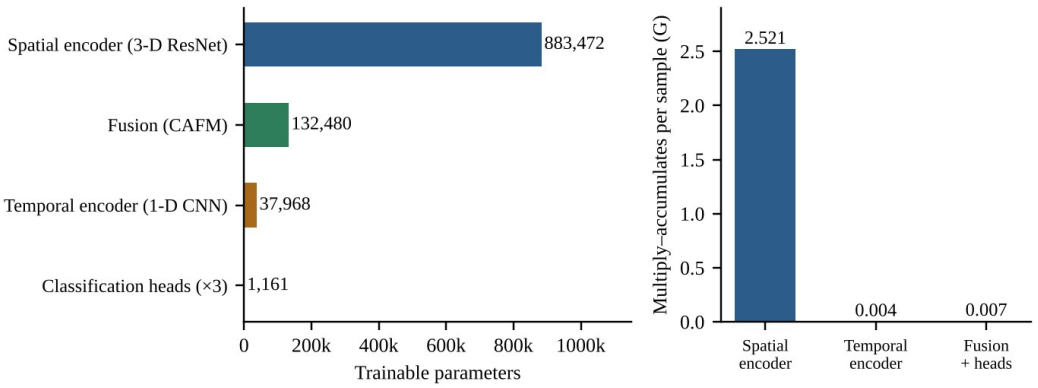


Figure 5.5: Left: trainable parameters by component. Right: multiply-accumulate operations per sample by component group.

The spatial encoder accounts for 83.7% of the parameters, the fusion module for 12.6%, the temporal encoder for 3.6% and the three heads for 0.11%. The attention-score tensor has 4,096 elements per sample (4 heads × 64 queries × 16 keys).

5.5 INTERFACE AND SHAPE VERIFICATION

The tensor contract of Table 3.1 is asserted by the test suite: the preprocessing outputs (1, 64, 64, 64) and (1, 500) at the default configuration; the spatial and temporal encoders return (B, 64, 128) and (B, 16, 128); the cross-attention module returns fused tokens (B, 64, 128) and attention weights (B, 4, 64, 16) whose rows sum to one; the model returns three logits tensors of shape (B, 3); Grad-CAM returns (B, D, H, W) values in [0, 1]; and the attention heatmap has shape (64, 16). A scaled-down configuration (the `tiny_config` fixture) is used for speed in the tests and exercises the same code.

5.6 SOFTWARE VERIFICATION RESULTS

The repository's recorded pytest node identifiers list 86 test functions in 12 files (78 unit and 8 integration tests; Figure 5.6, left), matching the count in the project

documentation. The tests could not be re-executed in the environment used to prepare this report (Section 5.1), so pass/fail status is not independently re-verified here. Statement coverage was reconstructed from the recorded coverage data file of the development machine by matching the executed line numbers against the executable statements of each source file. The total was 521 of 548 statements (95.1%), consistent with the 95% reported in the project documentation. Table 5.6 gives the per-module values; they may differ from the values printed by the coverage tool by a statement or so, because the executable-statement set was derived from the syntax tree rather than from the tool's own parser.

Table 5.6: Statement coverage by module (reconstructed from the recorded coverage data).

Module	Statements	Executed	Coverage (%)
src/config.py	38	38	100.0
src/data/dataset.py	43	43	100.0
src/data/synthetic.py	62	62	100.0
src/explainability/grad_cam_3d.py	70	69	98.6
src/explainability/plot_attention.py	22	22	100.0
src/models/cafn.py	31	31	100.0
src/models/cross_attention.py	45	45	100.0
src/models/spatial_encoder.py	37	37	100.0
src/models/temporal_encoder.py	14	14	100.0
src/preprocessing/mri_transforms.py	42	41	97.6
src/preprocessing/signal_cleaner.py	51	49	96.1
src/train.py	93	70	75.3
Total	**548**	**521**	**95.1**

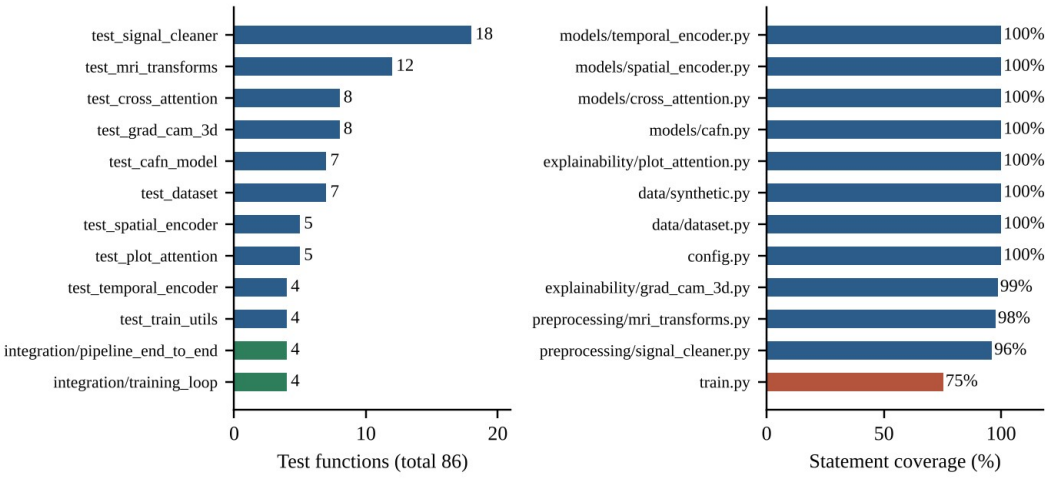


Figure 5.6: Left: number of test functions per test file (blue: unit; green: integration). Right: statement coverage per module (red: below 90%).

The lowest-covered module is `src/train.py` (75%). Its uncovered statements are mainly the body of the command-line `main()` function, which parses arguments, builds the synthetic dataset and writes a checkpoint, together with a few verbose-logging and edge-case branches.

5.7 TRAINING ACCEPTANCE CRITERIA

The training loop is verified by the four integration tests in `tests/integration/test_training_loop.py`, which train a scaled-down model on synthetic data. The configuration and the thresholds that they assert are given in Table 5.7. The tests assert thresholds, not point values; the exact loss and accuracy reached are therefore not part of the recorded results.

Table 5.7: Training-loop acceptance criteria asserted by `tests/integration/test_training_loop.py`.

Item	Specification
Data	24 synthetic samples (raw volume 40 × 48 × 32), validation split 25%, seed 0
Model	d_model 32, 2 heads, 8 spatial and 8 temporal tokens, FFN 64, dropout 0.1; MRI shape 32^3; VEP length 200
Training	15 epochs, batch size 4, Adam, learning rate 1e-3, weight decay 1e-4, auxiliary weight 0.3
Criterion 1	Final training loss < 0.7 × initial training loss
Criterion 2	Validation accuracy of the fused head > 0.5 (chance level for three balanced classes is 0.33)
Criterion 3	Training runs without a validation set (history of validation loss is empty)
Criterion 4	A saved and reloaded checkpoint reproduces the fused logits exactly (`assert_close`)

The four tests of `tests/integration/test_pipeline_end_to_end.py` exercise the raw-file path: synthetic NIfTI and CSV files are written and read through `PairedNeuroDataset`; the preprocessed tensors are passed through the model without shape errors; the loss is back-propagated and at least one parameter is shown to change after an optimiser step; Grad-CAM and attention plots are produced from a dataset sample; and the preprocessed Glaucoma and Optic Neuritis VEP peaks are shown to occur at different sample indices.

5.8 SUMMARY OF RESULTS

Table 5.8: Summary of the objective results.

Item	Result
H1: P100 latency preserved	149 of 150 synthetic trials within one sample (4 ms) of ground truth; one Glaucoma trial at 8 ms; mean errors between +0.00 and +0.80 ms
Mains attenuation	Mean 22.0 dB reduction of the 50 Hz component; notch −6 dB band 49.18–50.83 Hz
Foreground mask on synthetic MRI	Exactly 60.0% of voxels zeroed in every class; atrophy-proxy ROI fully zeroed in the Glaucoma volume
Model size	1,055,081 trainable parameters; about 2.53 G MACs per sample
Test suite	86 test functions (78 unit, 8 integration) in 12 files; not re-executed for this report
Statement coverage	95.1% overall; `src/train.py` 75%
H2: training acceptance	Thresholds defined and asserted by integration tests (loss < 70% of initial; validation accuracy > 0.5); measured values not reported here

Chapter 6
Discussion

6.1 INTRODUCTION

This chapter interprets the measurements of Chapter 5, relates them to the objectives, research questions and hypotheses of Section 1.4, compares the design with the literature of Chapter 2, and states the constraints under which the conclusions hold. The guiding principle is that every statement is limited to what the evidence supports: the project is a tested research prototype, and the evidence concerns software behaviour on synthetic data, not diagnostic performance.

6.2 INTERPRETATION OF THE RESULTS

6.2.1 SIGNAL CONDITIONING

The frequency response confirms the intended behaviour of the conditioning chain (Table 5.1): the pass-band is flat to better than 0.001 dB at 10 Hz, the 50 Hz component is removed by about 97 dB in the ideal response, and the −6 dB band of the notch is only 1.65 Hz wide, so that neighbouring frequencies are barely affected. The effective response at 1 Hz is about −5.8 dB, which is the −3 dB corner of the single-pass filter applied twice; the 1 Hz edge therefore removes baseline drift without cutting into the region around 10 Hz that carries the main VEP components.

The measured reduction of the injected 50 Hz component was 22 dB on average, much smaller than the 97 dB of the ideal response. This is not a failure of the filter. After filtering, the fitted 50 Hz amplitude (0.0125) is of the same order as the fitted amplitude at 47 Hz (0.0212), where nothing was injected: the residual is dominated by the broadband noise of the synthetic record, and the measured figure is therefore a lower bound on the notch's attenuation.

For the P100 latency, hypothesis H1 is supported at the level of the class means (errors of +0.64, +0.00 and +0.80 ms against a 4 ms sampling interval) and in 149 of 150 individual trials. The one exception, a Glaucoma record with an 8 ms error, is not specific to the zero-phase pipeline: the raw and single-pass results for the same class also reach a maximum error of 8 ms, so it is most likely due to the noise realisation and not to the filtering. The ordering of the three classes by detected latency (about

100, 108 and 141 ms) is preserved after conditioning, which is the property that matters for the downstream task.

A noteworthy observation is that the benefit of zero-phase over single-pass filtering cannot be demonstrated by these data. The mean difference in detected latency between the two implementations is below 0.5 ms in every class, i.e. much smaller than one sample, even though the single-pass filter has a group delay of 3 to 6 ms in the 10–30 Hz range (Table 5.3). The likely reason is that the synthetic P100 is a smooth, low-frequency peak whose maximum is insensitive to a small delay that is nearly constant across its bandwidth. The justification for zero-phase processing in this project therefore rests on the principle that latency is the clinical quantity (Odom et al., 2016) and on the absence of any cost to using it on offline records, not on an empirical gain at the project's sampling rate. At a higher sampling rate, or for sharper real waveforms, the difference could become measurable.

6.2.2 SYNTHETIC DATA AND THE FOREGROUND MASK

The class-conditioned generators behave as specified: the mean intensity of the atrophy region in the Glaucoma volume is 40 against about 100 elsewhere, the lesion region in the Optic Neuritis / CVI volume is about 180, and the Healthy Control volume is homogeneous (Table 5.4). The synthetic VEP classes differ in latency and amplitude as intended (Table 5.2 and Figure 5.4).

The mask results expose a limitation of the placeholder preprocessing. By construction the 60th-percentile threshold zeroes 60% of the voxels in every volume, whatever their content. In the Glaucoma volume this removes the entire atrophy region, so that the structural signature survives only as a contiguous block of zeros and no longer as a reduced intensity, while the lesion region of the Optic Neuritis / CVI volume is retained almost completely (99.7%). The mask is therefore not neutral with respect to the classes. This does not affect the correctness of the software, which was designed with the mask isolated for replacement (Section 3.7), but it shows that conclusions about what the spatial encoder can learn from the synthetic MRI depend on the preprocessing and cannot be carried over to real, skull-stripped scans.

6.2.3 MODEL SIZE AND COST

With 1,055,081 parameters, the model is small by the standards of current deep-learning systems, and about 84% of the parameters sit in the 3D residual encoder, as

expected because the channel count reaches 128 in its last block. The compute is concentrated differently: because the early 3D convolutions operate on the high-resolution feature maps, the first residual block alone accounts for 54% of the 2.53 billion MACs and the first two residual blocks for 82% (Table 5.5, Figure 5.5). The fusion module, which is the conceptual core of the project, accounts for only 13% of the parameters and well under 1% of the compute. A practical consequence is that the cost of the attention mechanism is negligible compared with the cost of reading the volume, and that any reduction in training time should target the early 3D stages, for example by lowering the input resolution or the width of the first blocks.

6.2.4 VERIFICATION

The test suite covers every module, and the line coverage reconstructed from the recorded data is consistent with the 95% in the project documentation. Coverage measures execution, not correctness, so it is best read together with the content of the tests, which assert shapes, value ranges, normalisation of attention rows, determinism, error handling and end-to-end gradient flow (Section 4.9). The least-covered module is the training script's command-line entry point, which is the least critical for correctness. The training acceptance criteria are meaningful in that they test learning (loss reduction and above-chance accuracy) and not only execution, but as thresholds on a toy configuration they say nothing about performance on real data.

6.3 ASSESSMENT AGAINST THE OBJECTIVES, QUESTIONS AND HYPOTHESES

Table 6.1: Assessment of research questions and hypotheses against the evidence.

Item	Evidence	Assessment
RQ1: common token representation through one pipeline	Shape contract asserted by tests; end-to-end tests from raw NIfTI and CSV files (Sections 5.5, 5.6)	Supported (software level)
RQ2 / H1: P100 latency preserved	Class-mean errors below one sample; 149 of 150 trials within 4 ms (Table 5.2)	Supported on synthetic data
RQ3 / H2: end-to-end optimisation and cost	Loss-reduction and accuracy thresholds asserted by tests; 1.06 M parameters, 2.53 G MACs (Tables 5.5, 5.7)	Supported by test design; measured curves not reported
RQ4: explanations from the same forward pass	Tests produce Grad-CAM maps and a 64 × 16 attention heatmap from a dataset sample (Section 5.6)	Supported (artefacts produced); faithfulness not assessed
H3: single shape contract	Shape tests across all modules; one configuration module (Section 4.8)	Supported
Diagnostic accuracy on real patients	No labelled clinical data	Not assessed

Item	Evidence	Assessment
Benefit of cross-attention over simpler fusion	No baseline in the repository	Not assessed

6.4 DESIGN OBSERVATIONS

6.4.1 COARSE TEMPORAL TOKENS

With 16 temporal tokens over a 2.0 s record, each token nominally covers 125 ms. The synthetic P100 peaks of the Healthy Control (100 ms) and Glaucoma (108 ms) classes both fall in the first token, while the delayed peak of Optic Neuritis / CVI (140 ms) falls in the second (Figure 4.2). The latency difference between the first two classes (8 ms) is therefore far below the nominal token resolution. This is not necessarily a defect, because the tokens are learned, overlapping features with a large receptive field and not literal time bins, but it means that the temporal tokens should not be read as calibrated time axes, and the attention heatmap cannot be read as pointing at a millisecond (Section 4.7).

6.4.2 SYNTHETIC SEPARABILITY

The synthetic classes differ in effects that are large relative to the noise (a 60% intensity reduction, an 80% intensity increase, shifts of up to 40 ms and a 2.4-fold amplitude change). A model that separates them has shown that the pipeline can learn from signals of this type, not that it can resolve the subtle, overlapping and heterogeneous changes present in patients. The same applies to the test thresholds. For this reason accuracy on this task is deliberately not presented as a result.

6.4.3 MULTI-TASK SUPERVISION AND ATTENTION

The auxiliary unimodal heads give each encoder its own gradient signal and so guard against the fused head ignoring one modality, a risk that is known for multimodal networks; whether they help in practice is an ablation that the repository does not contain. Likewise, the cross-attention weights are returned for inspection, but attention weights do not automatically constitute an explanation (Jain & Wallace, 2019; Wiegreffe & Pinter, 2019), and gradient-based maps can look plausible even when insensitive to the model's parameters (Adebayo et al., 2018). The explainability outputs of this project are accordingly presented as inspection aids whose faithfulness has not been tested.

6.5 COMPARISON WITH THE LITERATURE

The design follows the established practice of intermediate fusion of learned features (Baltrušaitis et al., 2019; Ramachandram & Taylor, 2017) and uses cross-attention in the sense of the multimodal transformer and co-attentional models (Tsai et al., 2019; Lu et al., 2019), with queries from the structural stream and keys and values from the electrophysiological stream. It differs from those works in the modalities (a volume and a short waveform, not language and vision), in scale (about 1 million parameters, compared with the large pre-trained models in that literature), and, most importantly, in evidence: those studies report results on benchmark datasets, whereas this project reports engineering verification on synthetic data. No numerical comparison with published classification results is therefore made or attempted, since it would not be like-for-like.

The preprocessing choices are in line with the recommendations of the VEP standard on the importance of latency (Odom et al., 2016) and with the signal-processing literature on zero-phase filtering (Gustafsson, 1996; Oppenheim & Schafer, 2010). The use of an intensity-percentile mask, by contrast, is below the standard of the neuroimaging literature, where dedicated tools such as HD-BET and SynthStrip are used (Isensee et al., 2019; Hoopes et al., 2022); the project acknowledges this explicitly.

6.6 CONSTRAINTS AND THREATS TO VALIDITY

Table 6.2: Constraints of the study and their consequence for the conclusions.

Constraint	Consequence
Synthetic data only; no clinical cohort	No statement about diagnostic accuracy, sensitivity, specificity or generalisation to patients is possible.
Training could not be executed for this report (the deep-learning libraries could not be installed in the environment used)	No measured training curves or validation metrics are reported; test thresholds are given instead. The tests were not re-run, so their pass status rests on the project's own records.
Single-seed, single-configuration measurements	The conditioning statistics are averaged over 150 seeded records, but no confidence intervals for the training behaviour exist.
Analytic complexity counts	Counts of parameters and MACs are derived from the layer definitions, not profiled; memory use and run time were not measured.
Coverage reconstructed from the recorded data file	Per-module values may differ from the tool's own report by a statement or so; the total agrees with the documented 95%.

Constraint	Consequence
Placeholder foreground mask	Conclusions about the spatial encoder depend on the mask and cannot be transferred to skull-stripped real scans.
No baselines or ablations	The added value of cross-attention and of the auxiliary heads is untested.

Chapter 7
Conclusion, Recommendations and Future Work

7.1 CONCLUSION

This project designed, implemented and verified a research prototype of a cross-attention fusion network (CAFN) that classifies paired 3D structural MRI and 1D visual evoked potentials into Healthy Control, Glaucoma and Optic Neuritis / Cortical Visual Impairment, and that produces 3D Grad-CAM maps and a spatial–temporal attention heatmap for each prediction. The work resolves the engineering problem posed in Section 1.3: the two modalities are brought to a common token space through a single configuration-driven pipeline, fused by letting 64 spatial tokens query 16 temporal tokens, and supervised by a fused head and two auxiliary unimodal heads. The pipeline can be exercised from raw NIfTI and CSV files through to an optimiser step and the explainability outputs, and it is covered by 86 automated tests with about 95% statement coverage.

What the project does *not* establish is equally important. Because no public dataset provides the required diagnosis labels, all learning experiments use a class-conditioned synthetic generator. The project therefore makes no claim about diagnostic accuracy, and no classification metric is presented as evidence. Its value is as a transparent, tested foundation on which an evaluation with real, labelled data can be built.

7.2 CONCLUSIONS AGAINST THE OBJECTIVES

Table 7.1: Status of each objective.

Objective	Outcome	Evidence
O1 Data interface	Achieved	Manifest-driven dataset yielding (1, 64, 64, 64) and (1, 500) tensors; end-to-end tests from raw files (Section 3.6, Section 5.5)
O2 Signal conditioning	Achieved for the VEP; MRI masking is a placeholder	Zero-phase pipeline keeps P100 latency within one sample in 149 of 150 trials; notch −6 dB band 1.65 Hz; percentile mask is not skull-stripping (Sections 5.2, 5.3)
O3 Dual encoders	Achieved	3D residual encoder (883,472 parameters) and 1D CNN (37,968) produce (B, 64, 128) and (B, 16, 128) tokens (Section 4.2, 4.3, Table 5.5)
O4 Cross-attention fusion	Achieved at the implementation level	Multi-head cross-attention with three heads and the multi-task loss; training acceptance criteria asserted by tests; no measured performance (Sections 4.4–4.6, 5.7)

Objective	Outcome	Evidence
O5 Explainability	Achieved at the implementation level	3D Grad-CAM on the fused logit and a 64 × 16 attention heatmap; faithfulness not evaluated (Section 4.7)
O6 Verification	Achieved	86 tests; 95.1% reconstructed coverage; filter, data and complexity measurements (Chapter 5)
O7 Honest scoping	Achieved	Limitations stated in Section 1.6, Chapter 5 and Section 6.6; excluded results listed

7.3 RECOMMENDATIONS

The recommendations below are ordered by priority. Priority 1 items are prerequisites for any claim about the usefulness of the system, Priority 2 items raise the scientific quality of such a claim, and Priority 3 items improve robustness and usability. Effort estimates are indicative for a small research team.

Table 7.2: Prioritised recommendations.

Priority	Recommendation	Basis in the findings	Indicative effort
1	Obtain a labelled, paired MRI–VEP cohort (or a clinical partner to label a subset), with ethics approval, diagnostic criteria and patient-level identifiers	Public data lack the diagnosis labels (Section 2.8); all current results are synthetic	Months; dependent on partners
1	Run the training end-to-end in an environment with PyTorch, MONAI and NiBabel installed, and record the loss and accuracy curves, the seed and the configuration for every run	Training curves could not be reported (Section 6.6)	Days
1	Add unimodal (MRI-only, VEP-only) and concatenation-fusion baselines, and ablations of the auxiliary-loss weight and of the number of tokens	The benefit of cross-attention is untested (Table 6.1)	1–2 weeks
2	Replace the percentile mask with validated skull-stripping (for example HD-BET or SynthStrip) as an optional step, and add registration or quality-control checks	The mask zeroes the atrophy region and is class-dependent (Section 6.2.2)	1–2 weeks
2	Adopt patient-level splits and cross-validation; report sensitivity, specificity, area under the ROC curve, confidence intervals and calibration, not accuracy alone	Only toy thresholds are asserted now	1–2 weeks
2	Test the explanation outputs: parameter-randomisation sanity checks for Grad-CAM, and comparison of the attention and saliency maps with clinician-marked regions	Attention and saliency are not automatically faithful (Section 6.4.3)	2–4 weeks

Priority	Recommendation	Basis in the findings	Indicative effort
3	Expose the real-data dataset through the command-line interface, add handling of a missing modality, and raise the coverage of the training entry point	The CLI only reads synthetic data; `src/train.py` is the least-covered module (Table 5.6)	1 week
3	Reduce the cost of the early 3D stages (lower input resolution, mixed precision, gradient checkpointing) if memory or time limits training	Most MACs are in the first residual blocks (Table 5.5)	1 week
3	Prepare a model card and data statement, and consider regulatory requirements early if clinical use is intended	The system is a research prototype	Ongoing

7.4 FUTURE WORK

- **Real-data training and external validation.** Train on the labelled cohort, validate on an independent site, and report performance with uncertainty.
- **Anatomically grounded tokens.** Register spatial features to named structures of the visual pathway (optic nerve, lateral geniculate nucleus, visual cortex) so that attention can be reported against anatomy, and align temporal tokens with the VEP landmarks N75, P100 and N135.
- **Richer electrophysiology.** Extend the temporal stream to multi-channel VEP, pattern electroretinography or full-field stimuli, which could allow localisation along the pathway.
- **Uncertainty and abstention.** Add calibrated probabilities and the option to defer ambiguous cases to a clinician.
- **Longitudinal modelling.** Use repeated MRI and VEP over time to model progression in glaucoma and recovery in optic neuritis.
- **Alternative fusion mechanisms.** Compare cross-attention with gating, bilinear pooling and bidirectional (two-way) cross-attention under the same training protocol.

7.5 CLOSING REMARKS

The prototype shows that a clean interface between a volumetric and a one-dimensional modality, a fusion module that makes the interaction explicit, and a testing discipline that reaches from file readers to explanation plots can be assembled in a compact, readable code base. The next stage of work is not another architecture

but data: with a labelled cohort, the baselines and the sanity checks listed above would turn this prototype into an experiment whose results can be believed.

REFERENCES

Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). Sanity checks for saliency maps. *Advances in Neural Information Processing Systems, 31*, 9505–9515.

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer normalization* (arXiv:1607.06450). arXiv. https://arxiv.org/abs/1607.06450

Baltrušaitis, T., Ahuja, C., & Morency, L.-P. (2019). Multimodal machine learning: A survey and taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *41*(2), 423–443. https://doi.org/10.1109/TPAMI.2018.2798607

Benson, N. C., Jamison, K. W., Arcaro, M. J., Vu, A. T., Glasser, M. F., Coalson, T. S., Van Essen, D. C., Yacoub, E., Ugurbil, K., Winawer, J., & Kay, K. (2018). The Human Connectome Project 7 Tesla retinotopy dataset: Description and population receptive field analysis. *Journal of Vision, 18*(13), Article 23. https://doi.org/10.1167/18.13.23

Butterworth, S. (1930). On the theory of filter amplifiers. *Experimental Wireless and the Wireless Engineer, 7*, 536–541.

Calhoun, V. D., & Sui, J. (2016). Multimodal fusion of brain imaging data: A key to finding the missing link(s) in complex mental illness. *Biological Psychiatry: Cognitive Neuroscience and Neuroimaging*, *1*(3), 230–244. https://doi.org/10.1016/j.bpsc.2015.12.005

Cardoso, M. J., Li, W., Brown, R., Ma, N., Kerfoot, E., Wang, Y., Murrey, B., Myronenko, A., Zhao, C., Yang, D., Nath, V., He, Y., Xu, Z., Hatamizadeh, A., Zhu, W., Liu, Y., Zheng, M., Tang, Y., Yang, I., . . . Feng, A. (2022). *MONAI: An open-source framework for deep learning in healthcare* (arXiv:2211.02701). arXiv. https://arxiv.org/abs/2211.02701

Çiçek, Ö., Abdulkadir, A., Lienkamp, S. S., Brox, T., & Ronneberger, O. (2016). 3D U-Net: Learning dense volumetric segmentation from sparse annotation. In S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, & W. Wells (Eds.), *Medical image computing and computer-assisted intervention – MICCAI 2016* (pp. 424–432). Springer. https://doi.org/10.1007/978-3-319-46723-8_49

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16×16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*. https://arxiv.org/abs/2010.11929

Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. C., Mark, R. G., Mietus, J. E., Moody, G. B., Peng, C.-K., & Stanley, H. E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research

resource for complex physiologic signals. *Circulation, 101*(23), e215–e220. https://doi.org/10.1161/01.CIR.101.23.e215

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience, 7*, Article 267. https://doi.org/10.3389/fnins.2013.00267

Gupta, N., Ang, L.-C., Noël de Tilly, L., Bidaisee, L., & Yücel, Y. H. (2006). Human glaucoma and neural degeneration in intracranial optic nerve, lateral geniculate nucleus, and visual cortex. *British Journal of Ophthalmology, 90*(6), 674–678. https://doi.org/10.1136/bjo.2005.086769

Gustafsson, F. (1996). Determining the initial states in forward-backward filtering. *IEEE Transactions on Signal Processing, 44*(4), 988–992. https://doi.org/10.1109/78.492552

Halliday, A. M., McDonald, W. I., & Mushin, J. (1972). Delayed visual evoked response in optic neuritis. *The Lancet, 299*(7758), 982–985. https://doi.org/10.1016/S0140-6736(72)91155-5

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Hoopes, A., Mora, J. S., Dalca, A. V., Fischl, B., & Hoffmann, M. (2022). SynthStrip: Skull-stripping for any brain image. *NeuroImage, 260*, Article 119474. https://doi.org/10.1016/j.neuroimage.2022.119474

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning, 37*, 448–456.

Isensee, F., Schell, M., Pflueger, I., Brugnara, G., Bonekamp, D., Neuberger, U., Wick, A., Schlemmer, H.-P., Heiland, S., Wick, W., Bendszus, M., Maier-Hein, K. H., & Kickingereder, P. (2019). Automated brain extraction of multisequence MRI using artificial neural networks. *Human Brain Mapping, 40*(17), 4952–4964. https://doi.org/10.1002/hbm.24750

Jain, S., & Wallace, B. C. (2019). Attention is not explanation. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 3543–3556. https://doi.org/10.18653/v1/N19-1357

Kamnitsas, K., Ledig, C., Newcombe, V. F. J., Simpson, J. P., Kane, A. D., Menon, D. K., Rueckert, D., & Glocker, B. (2017). Efficient multi-scale 3D CNN with fully connected CRF for accurate brain lesion segmentation. *Medical Image Analysis, 36,* 61–78. https://doi.org/10.1016/j.media.2016.10.004

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

LaMontagne, P. J., Benzinger, T. L. S., Morris, J. C., Keefe, S., Hornbeck, R., Xiong, C., Grant, E., Hassenstab, J., Moulder, K., Vlassenko, A. G., Raichle, M. E., Cruchaga, C., & Marcus, D. (2019). *OASIS-3: Longitudinal neuroimaging, clinical, and cognitive dataset for normal aging and Alzheimer disease* [Preprint]. medRxiv. https://doi.org/10.1101/2019.12.13.19014902

Lu, J., Batra, D., Parikh, D., & Lee, S. (2019). ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. *Advances in Neural Information Processing Systems, 32*, 13–23.

NiBabel Developers. (n.d.). *NiBabel: Access a cacophony of neuro-imaging file formats* [Computer software documentation]. https://nipy.org/nibabel/

Odom, J. V., Bach, M., Brigell, M., Holder, G. E., McCulloch, D. L., Mizota, A., & Tormene, A. P. (2016). ISCEV standard for clinical visual evoked potentials: (2016 update). *Documenta Ophthalmologica, 133*(1), 1–9. https://doi.org/10.1007/s10633-016-9553-y

OpenNeuro. (n.d.-a). *Dataset ds003787* [Data set]. https://openneuro.org/datasets/ds003787

OpenNeuro. (n.d.-b). *Dataset ds004496* [Data set]. https://openneuro.org/datasets/ds004496

Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-time signal processing* (3rd ed.). Pearson.

Parisi, V., Miglior, S., Manni, G., Centofanti, M., & Bucci, M. G. (2006). Clinical ability of pattern electroretinograms and visual evoked potentials in detecting visual dysfunction in ocular hypertension and glaucoma. *Ophthalmology, 113*(2), 216–228. https://doi.org/10.1016/j.ophtha.2005.10.044

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S.

(2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*, 8024–8035.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Ramachandram, D., & Taylor, G. W. (2017). Deep multimodal learning: A survey on recent advances and trends. *IEEE Signal Processing Magazine, 34*(6), 96–108. https://doi.org/10.1109/MSP.2017.2738401

Sakki, H. E. A., Dale, N. J., Sargent, J., Perez-Roche, T., & Bowman, R. (2018). Is there consensus in defining childhood cerebral visual impairment? A systematic review of terminology and definitions. *British Journal of Ophthalmology, 102*(4), 424–432. https://doi.org/10.1136/bjophthalmol-2017-310694

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision*, 618–626. https://doi.org/10.1109/ICCV.2017.74

Tham, Y.-C., Li, X., Wong, T. Y., Quigley, H. A., Aung, T., & Cheng, C.-Y. (2014). Global prevalence of glaucoma and projections of glaucoma burden through 2040: A systematic review and meta-analysis. *Ophthalmology, 121*(11), 2081–2090. https://doi.org/10.1016/j.ophtha.2014.05.013

Tsai, Y.-H. H., Bai, S., Liang, P. P., Kolter, J. Z., Morency, L.-P., & Salakhutdinov, R. (2019). Multimodal transformer for unaligned multimodal language sequences. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 6558–6569. https://doi.org/10.18653/v1/P19-1656

Van Essen, D. C., Smith, S. M., Barch, D. M., Behrens, T. E. J., Yacoub, E., & Ugurbil, K. (2013). The WU-Minn Human Connectome Project: An overview. *NeuroImage, 80*, 62–79. https://doi.org/10.1016/j.neuroimage.2013.05.041

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*, 5998–6008.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., . . . SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wiegreffe, S., & Pinter, Y. (2019). Attention is not not explanation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 11–20. https://doi.org/10.18653/v1/D19-1002

Appendix 1

Default Configuration

A1.1 CONFIGURATION PARAMETERS

All shapes and hyperparameters are defined in `src/config.py`. Table A1.1 summarises the default values used throughout the report; the full source is reproduced in Appendix 2.

Table A1.1: Default configuration values (`src/config.py`).

Group	Parameter	Default value
MRI	target_shape / target_spacing / orientation / foreground_percentile	(64, 64, 64) / (1.0, 1.0, 1.0) mm / RAS / 60
VEP	length / sample_rate / bandpass / notch_freq / notch_quality	500 / 250 Hz / 1–100 Hz / 50 Hz / 30
Model	d_model / num_heads / spatial_tokens / temporal_tokens	128 / 4 / 64 / 16
Model	ffn_hidden / dropout / num_classes / spatial_pool	256 / 0.1 / 3 / (4, 4, 4)
Training	batch_size / epochs / lr / weight_decay	8 / 20 / 1e-3 / 1e-4
Training	aux_loss_weight / val_split / seed / checkpoint_dir	0.3 / 0.2 / 42 / checkpoints

Appendix 2
Source Code Listings

A2.1 OVERVIEW

The listings below reproduce the source files of the `src/` package verbatim from the repository (commit `87362ec`). Line wrapping is introduced by the page width. The test suite is available in the repository under `tests/`.

Table A2.1: Source files reproduced in this appendix.

File	Purpose
`src/config.py`	Central configuration (shape contract and hyperparameters).
`src/data/synthetic.py`	Class-conditioned synthetic MRI and VEP generators.
`src/data/dataset.py`	Manifest-driven and synthetic PyTorch datasets.
`src/preprocessing/mri_transforms.py`	MRI loading, resampling, masking, scaling and padding or cropping.
`src/preprocessing/signal_cleaner.py`	VEP validation, zero-phase filtering, resampling and normalisation.
`src/models/spatial_encoder.py`	3D residual CNN producing spatial tokens.
`src/models/temporal_encoder.py`	1D CNN producing temporal tokens.
`src/models/cross_attention.py`	Multi-head cross-attention fusion module.
`src/models/cafn.py`	The complete CAFN model and the multi-task loss.
`src/train.py`	Training loop, evaluation, checkpointing and command-line entry point.
`src/explainability/grad_cam_3d.py`	3D Grad-CAM on the fused logit.
`src/explainability/plot_attention.py`	Spatial–temporal attention heatmap.

A2.2 LISTINGS

Listing A2.1: src/config.py

```
"""Central configuration for shapes, hyperparameters, and class labels.

Keeping these in one place guarantees the preprocessing stream, both
encoders, the fusion module, and the training loop all agree on tensor
shapes without hardcoding magic numbers in multiple files.
"""

from dataclasses import dataclass, field


CLASS_NAMES = ("Healthy Control", "Glaucoma", "Optic Neuritis / CVI")


@dataclass(frozen=True)
class MRIConfig:
    target_shape: tuple[int, int, int] = (64, 64, 64)
    target_spacing: tuple[float, float, float] = (1.0, 1.0, 1.0)
    orientation: str = "RAS"
    foreground_percentile: float = 60.0  # intensity-threshold skull-strip approx
```

```
@dataclass(frozen=True)
class VEPConfig:
    length: int = 500
    sample_rate: float = 250.0  # Hz -> 500 samples spans 2.0s per ISCEV sweep
    bandpass_low: float = 1.0
    bandpass_high: float = 100.0
    notch_freq: float = 50.0
    notch_quality: float = 30.0


@dataclass(frozen=True)
class ModelConfig:
    d_model: int = 128
    num_heads: int = 4
    spatial_tokens: int = 64   # 4x4x4 adaptive pool
    temporal_tokens: int = 16
    spatial_pool: tuple[int, int, int] = (4, 4, 4)
    ffn_hidden: int = 256
    dropout: float = 0.1
    num_classes: int = len(CLASS_NAMES)


@dataclass(frozen=True)
class TrainConfig:
    batch_size: int = 8
    epochs: int = 20
    lr: float = 1e-3
    weight_decay: float = 1e-4
    aux_loss_weight: float = 0.3
    val_split: float = 0.2
    seed: int = 42
    checkpoint_dir: str = "checkpoints"


@dataclass(frozen=True)
class Config:
    mri: MRIConfig = field(default_factory=MRIConfig)
    vep: VEPConfig = field(default_factory=VEPConfig)
    model: ModelConfig = field(default_factory=ModelConfig)
    train: TrainConfig = field(default_factory=TrainConfig)


DEFAULT_CONFIG = Config()
```

Listing A2.2: src/data/synthetic.py

```
"""Synthetic MRI + VEP generators.

The datasets named in Project.md (OpenNeuro, HCP Retinotopy, OASIS, MNE
sample, PhysioNet) provide structural/functional scans and generic
electrophysiology, but none carry the clinical Healthy/Glaucoma/Optic
Neuritis-CVI labels this model targets -- that requires a curated clinical
cohort. This module generates class-conditioned synthetic MRI volumes and
VEP waveforms with the same interface real data would have, so the rest of
the system (preprocessing, model, training, explainability) can be built,
exercised, and tested end-to-end while a labeled cohort is sourced
separately.

Class-conditioning is deliberately simplistic but structured so a model can
learn from it: label 0 (Healthy) is unstructured background; label 1
(Glaucoma) has focal MRI intensity loss (atrophy proxy) and reduced VEP
amplitude; label 2 (Optic Neuritis / CVI) has focal MRI hyperintensity
(lesion proxy) and delayed VEP P100 latency (the clinical hallmark of optic
neuritis).
"""

import csv
import os

import nibabel as nib
import numpy as np

from src.config import Config, DEFAULT_CONFIG

NUM_CLASSES = DEFAULT_CONFIG.model.num_classes
```

```
def synthetic_mri_volume(
    label: int, shape: tuple[int, int, int] = (80, 96, 64), rng: np.random.Generator |
   None = None
) -> np.ndarray:
    """Generate a class-conditioned synthetic 3D MRI-like volume."""
    rng = rng or np.random.default_rng()
    volume = rng.normal(loc=100.0, scale=15.0, size=shape).clip(min=0)
    d, h, w = shape
    if label == 1:  # Glaucoma: focal atrophy (intensity loss)
        volume[d // 2 :, : h // 2, : w // 2] *= 0.4
    elif label == 2:  # Optic Neuritis / CVI: focal lesion (hyperintensity)
        volume[: d // 2, h // 2 :, w // 2 :] *= 1.8
    return volume.astype(np.float32)


def synthetic_vep_signal(
    label: int, fs: float = 250.0, duration_s: float = 2.0, rng: np.random.Generator |
   None = None
) -> np.ndarray:
    """Generate a class-conditioned synthetic VEP waveform (N75-P100-N145)."""
    rng = rng or np.random.default_rng()
    t = np.arange(0.0, duration_s, 1.0 / fs)

    if label == 0:  # Healthy
        latency, amplitude = 0.100, 6.0
    elif label == 1:  # Glaucoma: reduced amplitude, near-normal latency
        latency, amplitude = 0.108, 2.5
    else:  # Optic Neuritis / CVI: delayed P100 latency
        latency, amplitude = 0.140, 4.0

    p100 = amplitude * np.exp(-((t - latency) ** 2) / (2 * 0.012**2))
    n75 = -0.5 * amplitude * np.exp(-((t - (latency - 0.025)) ** 2) / (2 * 0.010**2))
    n145 = -0.4 * amplitude * np.exp(-((t - (latency + 0.045)) ** 2) / (2 * 0.015**2))
    waveform = p100 + n75 + n145

    drift = 0.3 * np.sin(2 * np.pi * 0.3 * t)
    mains = 0.15 * np.sin(2 * np.pi * 50.0 * t)
    emg_noise = rng.normal(0.0, 0.3, size=t.shape)

    return (waveform + drift + mains + emg_noise).astype(np.float64)


def write_synthetic_dataset(
    root: str,
    samples_per_class: int = 4,
    cfg: Config = DEFAULT_CONFIG,
    seed: int = 0,
) -> str:
    """Write synthetic NIfTI + VEP CSV files and a manifest.csv under `root`.

    Mirrors the data/raw/{mri,vep} layout described in Project.md. Returns
    the manifest CSV path, with columns: subject_id, mri_path, vep_path, fs, label.
    """
    rng = np.random.default_rng(seed)
    mri_dir = os.path.join(root, "mri")
    vep_dir = os.path.join(root, "vep")
    os.makedirs(mri_dir, exist_ok=True)
    os.makedirs(vep_dir, exist_ok=True)

    manifest_path = os.path.join(root, "manifest.csv")
    rows = []
    subject_idx = 0
    for label in range(NUM_CLASSES):
        for _ in range(samples_per_class):
            subject_id = f"sub-{subject_idx:04d}"
            volume = synthetic_mri_volume(label, rng=rng)
            affine = np.diag([1.0, 1.0, 1.0, 1.0])
            img = nib.Nifti1Image(volume, affine)
            mri_path = os.path.join(mri_dir, f"{subject_id}_mri.nii.gz")
            nib.save(img, mri_path)

            signal = synthetic_vep_signal(label, fs=cfg.vep.sample_rate, rng=rng)
            vep_path = os.path.join(vep_dir, f"{subject_id}_vep.csv")
            with open(vep_path, "w", newline="") as f:
                writer = csv.writer(f)
                writer.writerow(["amplitude"])
```

```
                for value in signal:
                    writer.writerow([value])

            rows.append(
                {
                    "subject_id": subject_id,
                    "mri_path": mri_path,
                    "vep_path": vep_path,
                    "fs": cfg.vep.sample_rate,
                    "label": label,
                }
            )
            subject_idx += 1

    with open(manifest_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["subject_id", "mri_path", "vep_path",
   "fs", "label"])
        writer.writeheader()
        writer.writerows(rows)

    return manifest_path
```

Listing A2.3: src/data/dataset.py

```
"""PyTorch Dataset classes for paired MRI + VEP samples.

`PairedNeuroDataset` reads a manifest CSV (subject_id, mri_path, vep_path,
fs, label) and applies the real preprocessing pipeline -- this is what a
labeled clinical cohort would plug into. `SyntheticNeuroDataset` generates
class-conditioned synthetic samples in-memory (no disk I/O) through the same
preprocessing pipeline, used for fast unit tests, integration tests, and
development/training-loop validation ahead of real labeled data.
"""

import csv

import numpy as np
import torch
from torch.utils.data import Dataset

from src.config import Config, DEFAULT_CONFIG
from src.data.synthetic import synthetic_mri_volume, synthetic_vep_signal
from src.preprocessing.mri_transforms import preprocess_mri_file,
   preprocess_mri_volume
from src.preprocessing.signal_cleaner import clean_vep_signal


class PairedNeuroDataset(Dataset):
    """Loads (MRI volume, VEP signal, label) triples from a manifest CSV."""

    def __init__(self, manifest_path: str, cfg: Config = DEFAULT_CONFIG):
        self.cfg = cfg
        with open(manifest_path, newline="") as f:
            self.rows = list(csv.DictReader(f))
        if not self.rows:
            raise ValueError(f"Manifest at {manifest_path} has no rows")

    def __len__(self) -> int:
        return len(self.rows)

    def __getitem__(self, idx: int) -> dict:
        row = self.rows[idx]
        mri = preprocess_mri_file(row["mri_path"], self.cfg.mri)

        raw_signal = np.loadtxt(row["vep_path"], delimiter=",", skiprows=1)
        fs = float(row.get("fs") or self.cfg.vep.sample_rate)
        vep = clean_vep_signal(raw_signal, fs, self.cfg.vep)

        return {
            "mri": torch.from_numpy(mri),
            "vep": torch.from_numpy(vep).unsqueeze(0),
            "label": torch.tensor(int(row["label"]), dtype=torch.long),
            "subject_id": row["subject_id"],
        }


class SyntheticNeuroDataset(Dataset):
```

```
    """In-memory class-conditioned synthetic dataset, real preprocessing applied."""

    def __init__(
        self,
        num_samples: int,
        cfg: Config = DEFAULT_CONFIG,
        seed: int = 0,
        raw_mri_shape: tuple[int, int, int] = (80, 96, 64),
    ):
        self.cfg = cfg
        self.num_samples = num_samples
        self.raw_mri_shape = raw_mri_shape
        self.rng = np.random.default_rng(seed)
        num_classes = cfg.model.num_classes
        # Balanced labels, deterministically shuffled.
        base_labels = [i % num_classes for i in range(num_samples)]
        self.rng.shuffle(base_labels)
        self.labels = base_labels

    def __len__(self) -> int:
        return self.num_samples

    def __getitem__(self, idx: int) -> dict:
        label = self.labels[idx]
        raw_volume = synthetic_mri_volume(label, shape=self.raw_mri_shape,
   rng=self.rng)
        mri = preprocess_mri_volume(raw_volume, zooms=(1.0, 1.0, 1.0),
   cfg=self.cfg.mri)

        raw_signal = synthetic_vep_signal(label, fs=self.cfg.vep.sample_rate,
   rng=self.rng)
        vep = clean_vep_signal(raw_signal, self.cfg.vep.sample_rate, self.cfg.vep)

        return {
            "mri": torch.from_numpy(mri),
            "vep": torch.from_numpy(vep).unsqueeze(0),
            "label": torch.tensor(label, dtype=torch.long),
            "subject_id": f"synthetic-{idx:04d}",
        }
```

Listing A2.4: src/preprocessing/mri_transforms.py

```
"""3D sMRI spatial standardization: RAS reorientation, 1mm isotropic
resampling, intensity-threshold foreground masking, and normalization.

Note on skull-stripping: true skull-stripping (FSL BET / HD-BET / SynthStrip)
requires an external binary or a pretrained segmentation network that is not
pip-installable. This module ships a lightweight intensity-percentile
foreground mask as a prototype-stage approximation -- it zeroes low-intensity
background/skull voxels without needing an external dependency. Swap in
HD-BET or SynthStrip for clinical-grade deployment; the rest of the pipeline
is agnostic to which foreground mask produced its input.
"""

import nibabel as nib
import numpy as np
from monai.transforms import Resize, ResizeWithPadOrCrop, ScaleIntensity

from src.config import MRIConfig, DEFAULT_CONFIG


def load_nifti(path: str) -> tuple[np.ndarray, np.ndarray, tuple[float, float,
   float]]:
    """Load a NIfTI volume, reoriented to RAS (nibabel's closest-canonical).

    Returns (data, affine, voxel_zooms).
    """
    img = nib.load(path)
    img = nib.as_closest_canonical(img)
    data = np.asarray(img.get_fdata(), dtype=np.float32)
    if data.ndim != 3:
        raise ValueError(f"Expected a 3D volume, got shape {data.shape}")
    zooms = tuple(float(z) for z in img.header.get_zooms()[:3])
    if any(z <= 0 for z in zooms):
        raise ValueError(f"Invalid voxel spacing in header: {zooms}")
    return data, img.affine, zooms
```

```
def foreground_mask_threshold(volume: np.ndarray, percentile: float) -> np.ndarray:
    """Zero out background/skull voxels below an intensity percentile.

    Percentile is computed over strictly-positive voxels so that an
    already-large air/background region does not collapse the threshold to
    zero.
    """
    nonzero = volume[volume > 0]
    if nonzero.size == 0:
        return volume
    threshold = np.percentile(nonzero, percentile)
    masked = volume.copy()
    masked[masked < threshold] = 0.0
    return masked


def resample_isotropic(
    volume: np.ndarray,
    zooms: tuple[float, float, float],
    target_spacing: tuple[float, float, float] = (1.0, 1.0, 1.0),
) -> np.ndarray:
    """Resample a volume to isotropic spacing by scaling voxel counts.

    MONAI's shape-based `Resize` combined with a zoom-derived target shape
    is mathematically equivalent to physical-spacing resampling and avoids
    MetaTensor/affine bookkeeping for a standalone array pipeline.
    """
    new_shape = tuple(
        max(1, round(dim * zoom / target))
        for dim, zoom, target in zip(volume.shape, zooms, target_spacing)
    )
    resize = Resize(spatial_size=new_shape, mode="trilinear", align_corners=False)
    resized = resize(volume[np.newaxis, ...])  # add channel dim
    return np.asarray(resized[0])


def preprocess_mri_volume(
    volume: np.ndarray,
    zooms: tuple[float, float, float],
    cfg: MRIConfig = DEFAULT_CONFIG.mri,
) -> np.ndarray:
    """Full spatial standardization pipeline for an already-RAS volume.

    Returns a float32 array of shape (1, *cfg.target_shape) scaled to [0, 1].
    """
    if volume.ndim != 3:
        raise ValueError(f"Expected a 3D volume, got shape {volume.shape}")

    isotropic = resample_isotropic(volume, zooms, cfg.target_spacing)
    masked = foreground_mask_threshold(isotropic, cfg.foreground_percentile)

    if masked.max() > masked.min():
        scaler = ScaleIntensity(minv=0.0, maxv=1.0)
        normalized = np.asarray(scaler(masked[np.newaxis, ...]))
    else:
        normalized = masked[np.newaxis, ...].astype(np.float32)

    pad_crop = ResizeWithPadOrCrop(spatial_size=cfg.target_shape)
    standardized = np.asarray(pad_crop(normalized))
    return standardized.astype(np.float32)


def preprocess_mri_file(path: str, cfg: MRIConfig = DEFAULT_CONFIG.mri) -> np.ndarray:
    """Convenience wrapper: load a NIfTI file and run full standardization."""
    data, _, zooms = load_nifti(path)
    return preprocess_mri_volume(data, zooms, cfg)
```

Listing A2.5: src/preprocessing/signal_cleaner.py

```
"""1D VEP signal conditioning: zero-phase Butterworth bandpass + mains notch.

Filtering order matters for VEP work: bandpass first removes drift and
high-frequency EMG jitter, then the notch removes the narrowband 50Hz mains
hum. Both filters use zero-phase (filtfilt / sosfiltfilt) application so the
P100 peak latency -- the clinically meaningful measurement -- is not
shifted in time by the filter's group delay.
```

```
"""

import numpy as np
from scipy.signal import butter, iirnotch, sosfiltfilt, filtfilt

from src.config import VEPConfig, DEFAULT_CONFIG


def _validate_signal(signal: np.ndarray) -> np.ndarray:
    signal = np.asarray(signal, dtype=np.float64)
    if signal.ndim != 1:
        raise ValueError(f"Expected a 1D signal, got shape {signal.shape}")
    if signal.size == 0:
        raise ValueError("Signal is empty")
    if not np.all(np.isfinite(signal)):
        raise ValueError("Signal contains NaN or Inf values")
    return signal


def butterworth_bandpass(
    signal: np.ndarray,
    fs: float,
    lowcut: float = 1.0,
    highcut: float = 100.0,
    order: int = 4,
) -> np.ndarray:
    """Zero-phase 4th-order Butterworth bandpass filter."""
    signal = _validate_signal(signal)
    nyquist = fs / 2.0
    if lowcut <= 0:
        raise ValueError("lowcut must be > 0")
    if highcut >= nyquist:
        raise ValueError(
            f"highcut ({highcut} Hz) must be below the Nyquist frequency "
            f"({nyquist} Hz) for fs={fs} Hz"
        )
    if lowcut >= highcut:
        raise ValueError("lowcut must be < highcut")

    sos = butter(order, [lowcut, highcut], btype="bandpass", fs=fs, output="sos")
    # Zero-phase filtering needs enough samples relative to the filter's
    # padding requirement; short synthetic signals can trip this up.
    min_len = 3 * (2 * len(sos) + 1)
    if signal.size <= min_len:
        raise ValueError(
            f"Signal too short ({signal.size} samples) for zero-phase "
            f"filtering; need > {min_len} samples"
        )
    return sosfiltfilt(sos, signal)


def notch_filter(
    signal: np.ndarray,
    fs: float,
    freq: float = 50.0,
    quality: float = 30.0,
) -> np.ndarray:
    """Zero-phase IIR notch filter to remove AC power-line hum."""
    signal = _validate_signal(signal)
    nyquist = fs / 2.0
    if freq >= nyquist:
        raise ValueError(
            f"notch freq ({freq} Hz) must be below Nyquist ({nyquist} Hz)"
        )
    b, a = iirnotch(freq, quality, fs)
    return filtfilt(b, a, signal)


def zscore_normalize(signal: np.ndarray, eps: float = 1e-8) -> np.ndarray:
    """Zero mean, unit variance normalization."""
    signal = _validate_signal(signal)
    std = signal.std()
    return (signal - signal.mean()) / (std + eps)


def resample_to_length(signal: np.ndarray, target_length: int) -> np.ndarray:
    """Linear-interpolation resample to a fixed number of samples."""
```

```
    signal = _validate_signal(signal)
    if signal.size == target_length:
        return signal
    x_old = np.linspace(0.0, 1.0, num=signal.size)
    x_new = np.linspace(0.0, 1.0, num=target_length)
    return np.interp(x_new, x_old, signal)


def clean_vep_signal(
    raw_signal: np.ndarray,
    fs: float,
    cfg: VEPConfig = DEFAULT_CONFIG.vep,
) -> np.ndarray:
    """Full 1D conditioning pipeline: bandpass -> notch -> resample -> z-score.

    Returns a float32 array of shape (cfg.length,).
    """
    signal = _validate_signal(raw_signal)
    filtered = butterworth_bandpass(
        signal, fs, lowcut=cfg.bandpass_low, highcut=cfg.bandpass_high
    )
    denotched = notch_filter(filtered, fs, freq=cfg.notch_freq,
   quality=cfg.notch_quality)
    resampled = resample_to_length(denotched, cfg.length)
    normalized = zscore_normalize(resampled)
    return normalized.astype(np.float32)
```

Listing A2.6: src/models/spatial_encoder.py

```
"""3D-CNN / ResNet backbone: structural MRI volume -> spatial tokens."""

import torch
import torch.nn as nn

from src.config import ModelConfig, DEFAULT_CONFIG


class ResidualBlock3D(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv3d(in_channels, out_channels, 3, stride=stride, padding=1,
   bias=False)
        self.bn1 = nn.BatchNorm3d(out_channels)
        self.conv2 = nn.Conv3d(out_channels, out_channels, 3, stride=1, padding=1,
   bias=False)
        self.bn2 = nn.BatchNorm3d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        self.shortcut = None
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv3d(in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm3d(out_channels),
            )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = x if self.shortcut is None else self.shortcut(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return self.relu(out + identity)


class SpatialEncoder3D(nn.Module):
    """(B, 1, D, H, W) -> (B, spatial_tokens, d_model) spatial token sequence.

    `self.last_conv` exposes the final residual block so Grad-CAM3D can hook
    its output activation map.
    """

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model, in_channels: int = 1):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv3d(in_channels, 16, 3, stride=1, padding=1, bias=False),
            nn.BatchNorm3d(16),
            nn.ReLU(inplace=True),
        )
        self.layer1 = ResidualBlock3D(16, 32, stride=2)
```

```
        self.layer2 = ResidualBlock3D(32, 64, stride=2)
        self.layer3 = ResidualBlock3D(64, cfg.d_model, stride=2)
        self.last_conv = self.layer3

        self.pool = nn.AdaptiveAvgPool3d(cfg.spatial_pool)
        self.d_model = cfg.d_model

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.pool(x)  # (B, d_model, *spatial_pool)
        tokens = x.flatten(2).transpose(1, 2)  # (B, tokens, d_model)
        return tokens
```

Listing A2.7: src/models/temporal_encoder.py

```
"""1D-CNN latency backbone: VEP waveform -> temporal tokens."""

import torch
import torch.nn as nn

from src.config import ModelConfig, DEFAULT_CONFIG


class TemporalEncoder1D(nn.Module):
    """(B, 1, L) -> (B, temporal_tokens, d_model) temporal token sequence."""

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model, in_channels: int = 1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(in_channels, 16, kernel_size=7, stride=1, padding=3,
  bias=False),
            nn.BatchNorm1d(16),
            nn.ReLU(inplace=True),
            nn.Conv1d(16, 32, kernel_size=5, stride=2, padding=2, bias=False),
            nn.BatchNorm1d(32),
            nn.ReLU(inplace=True),
            nn.Conv1d(32, 64, kernel_size=5, stride=2, padding=2, bias=False),
            nn.BatchNorm1d(64),
            nn.ReLU(inplace=True),
            nn.Conv1d(64, cfg.d_model, kernel_size=3, stride=2, padding=1,
  bias=False),
            nn.BatchNorm1d(cfg.d_model),
            nn.ReLU(inplace=True),
        )
        self.pool = nn.AdaptiveAvgPool1d(cfg.temporal_tokens)
        self.d_model = cfg.d_model

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.net(x)          # (B, d_model, L')
        x = self.pool(x)         # (B, d_model, temporal_tokens)
        tokens = x.transpose(1, 2)  # (B, temporal_tokens, d_model)
        return tokens
```

Listing A2.8: src/models/cross_attention.py

```
"""Multi-Head Cross-Attention Fusion Module (CAFM).

Spatial 3D tokens are projected as Queries; temporal 1D latency tokens are
projected as Keys and Values, so each spatial brain region dynamically
queries the electrophysiological latency bins:

    Attention(Q, K, V) = Softmax(Q K^T / sqrt(d_k)) V
"""

import math

import torch
import torch.nn as nn

from src.config import ModelConfig, DEFAULT_CONFIG


class MultiHeadCrossAttention(nn.Module):
    """Q from `query_tokens`, K/V from `kv_tokens`. Sequence lengths may differ."""
```

```
    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model):
        super().__init__()
        if cfg.d_model % cfg.num_heads != 0:
            raise ValueError("d_model must be divisible by num_heads")
        self.num_heads = cfg.num_heads
        self.head_dim = cfg.d_model // cfg.num_heads
        self.d_model = cfg.d_model

        self.w_q = nn.Linear(cfg.d_model, cfg.d_model)
        self.w_k = nn.Linear(cfg.d_model, cfg.d_model)
        self.w_v = nn.Linear(cfg.d_model, cfg.d_model)
        self.out_proj = nn.Linear(cfg.d_model, cfg.d_model)
        self.dropout = nn.Dropout(cfg.dropout)

    def _split_heads(self, x: torch.Tensor) -> torch.Tensor:
        b, n, _ = x.shape
        return x.view(b, n, self.num_heads, self.head_dim).transpose(1, 2)  # (B, H,
   N, Dh)

    def forward(self, query_tokens: torch.Tensor, kv_tokens: torch.Tensor):
        b = query_tokens.shape[0]
        q = self._split_heads(self.w_q(query_tokens))
        k = self._split_heads(self.w_k(kv_tokens))
        v = self._split_heads(self.w_v(kv_tokens))

        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        attn = torch.softmax(scores, dim=-1)  # (B, H, Nq, Nk)
        attn = self.dropout(attn)
        context = torch.matmul(attn, v)  # (B, H, Nq, Dh)

        context = context.transpose(1, 2).contiguous().view(b, -1, self.d_model)
        out = self.out_proj(context)
        return out, attn


class CrossAttentionFusion(nn.Module):
    """CAFM: cross-attention -> residual + LayerNorm -> feed-forward block."""

    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model):
        super().__init__()
        self.cross_attn = MultiHeadCrossAttention(cfg)
        self.norm1 = nn.LayerNorm(cfg.d_model)
        self.ffn = nn.Sequential(
            nn.Linear(cfg.d_model, cfg.ffn_hidden),
            nn.ReLU(inplace=True),
            nn.Dropout(cfg.dropout),
            nn.Linear(cfg.ffn_hidden, cfg.d_model),
        )
        self.norm2 = nn.LayerNorm(cfg.d_model)

    def forward(self, spatial_tokens: torch.Tensor, temporal_tokens: torch.Tensor):
        attn_out, attn_weights = self.cross_attn(spatial_tokens, temporal_tokens)
        fused = self.norm1(spatial_tokens + attn_out)
        ffn_out = self.ffn(fused)
        fused = self.norm2(fused + ffn_out)
        return fused, attn_weights
```

Listing A2.9: src/models/cafn.py

```
"""Cross-Attention Fusion Network (CAFN): full model wiring.

Dual-stream encoders feed the cross-attention fusion module; a fused
classification head and two auxiliary unimodal heads train jointly
(multi-task supervision) so the network cannot collapse onto a single
dominant modality.
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

from src.config import ModelConfig, DEFAULT_CONFIG
from src.models.spatial_encoder import SpatialEncoder3D
from src.models.temporal_encoder import TemporalEncoder1D
from src.models.cross_attention import CrossAttentionFusion
```

```
class CAFN(nn.Module):
    def __init__(self, cfg: ModelConfig = DEFAULT_CONFIG.model):
        super().__init__()
        self.cfg = cfg
        self.spatial_encoder = SpatialEncoder3D(cfg)
        self.temporal_encoder = TemporalEncoder1D(cfg)
        self.fusion = CrossAttentionFusion(cfg)

        self.fused_head = nn.Linear(cfg.d_model, cfg.num_classes)
        self.spatial_head = nn.Linear(cfg.d_model, cfg.num_classes)
        self.temporal_head = nn.Linear(cfg.d_model, cfg.num_classes)

    def forward(self, mri_volume: torch.Tensor, vep_signal: torch.Tensor) -> dict:
        spatial_tokens = self.spatial_encoder(mri_volume)    # (B, S, d)
        temporal_tokens = self.temporal_encoder(vep_signal)  # (B, T, d)

        fused_tokens, attn_weights = self.fusion(spatial_tokens, temporal_tokens)

        fused_embedding = fused_tokens.mean(dim=1)
        spatial_embedding = spatial_tokens.mean(dim=1)
        temporal_embedding = temporal_tokens.mean(dim=1)

        return {
            "logits_fused": self.fused_head(fused_embedding),
            "logits_spatial": self.spatial_head(spatial_embedding),
            "logits_temporal": self.temporal_head(temporal_embedding),
            "attn_weights": attn_weights,  # (B, num_heads, S, T)
            "fused_tokens": fused_tokens,
            "spatial_tokens": spatial_tokens,
            "temporal_tokens": temporal_tokens,
        }

    def multi_task_loss(self, outputs: dict, labels: torch.Tensor, aux_weight: float =
  0.3):
        loss_fused = F.cross_entropy(outputs["logits_fused"], labels)
        loss_spatial = F.cross_entropy(outputs["logits_spatial"], labels)
        loss_temporal = F.cross_entropy(outputs["logits_temporal"], labels)
        total = loss_fused + aux_weight * (loss_spatial + loss_temporal)
        return total, {
            "loss_fused": loss_fused.item(),
            "loss_spatial": loss_spatial.item(),
            "loss_temporal": loss_temporal.item(),
            "loss_total": total.item(),
        }
```

Listing A2.10: src/train.py

```
"""Multi-task training loop for CAFN.

Trains the fused cross-attention head jointly with the two auxiliary
unimodal heads (`CAFN.multi_task_loss`). Callable as a library function
(`train_model`) for tests/notebooks, or as a script:

    python -m src.train --epochs 20 --synthetic-samples 64
"""

import argparse
import os

import torch
from torch.utils.data import DataLoader, Subset

from src.config import Config, DEFAULT_CONFIG
from src.data.dataset import SyntheticNeuroDataset
from src.models.cafn import CAFN


def train_val_split(dataset, val_split: float, seed: int) -> tuple[Subset, Subset]:
    n = len(dataset)
    n_val = max(1, int(n * val_split)) if n > 1 else 0
    generator = torch.Generator().manual_seed(seed)
    perm = torch.randperm(n, generator=generator).tolist()
    val_idx, train_idx = perm[:n_val], perm[n_val:]
    if not train_idx:
        train_idx, val_idx = perm, []
    return Subset(dataset, train_idx), Subset(dataset, val_idx)
```

```
@torch.no_grad()
def evaluate(model: CAFN, loader: DataLoader, device: torch.device) -> dict:
    model.eval()
    total, correct, loss_sum = 0, 0, 0.0
    for batch in loader:
        mri = batch["mri"].to(device)
        vep = batch["vep"].to(device)
        labels = batch["label"].to(device)
        outputs = model(mri, vep)
        loss, _ = model.multi_task_loss(outputs, labels)
        preds = outputs["logits_fused"].argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        loss_sum += loss.item() * labels.size(0)
    if total == 0:
        return {"loss": float("nan"), "accuracy": float("nan")}
    return {"loss": loss_sum / total, "accuracy": correct / total}


def train_model(
    model: CAFN,
    train_dataset,
    val_dataset=None,
    cfg: Config = DEFAULT_CONFIG,
    device: torch.device | None = None,
    verbose: bool = False,
) -> dict:
    """Runs the multi-task training loop. Returns per-epoch history."""
    device = device or torch.device("cpu")
    model.to(device)

    train_loader = DataLoader(train_dataset, batch_size=cfg.train.batch_size,
   shuffle=True)
    val_loader = (
        DataLoader(val_dataset, batch_size=cfg.train.batch_size, shuffle=False)
        if val_dataset is not None and len(val_dataset) > 0
        else None
    )

    optimizer = torch.optim.Adam(
        model.parameters(), lr=cfg.train.lr, weight_decay=cfg.train.weight_decay
    )

    history = {"train_loss": [], "val_loss": [], "val_accuracy": []}
    for epoch in range(cfg.train.epochs):
        model.train()
        epoch_loss, n_samples = 0.0, 0
        for batch in train_loader:
            mri = batch["mri"].to(device)
            vep = batch["vep"].to(device)
            labels = batch["label"].to(device)

            optimizer.zero_grad()
            outputs = model(mri, vep)
            loss, _ = model.multi_task_loss(outputs, labels,
   aux_weight=cfg.train.aux_loss_weight)
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item() * labels.size(0)
            n_samples += labels.size(0)

        train_loss = epoch_loss / max(n_samples, 1)
        history["train_loss"].append(train_loss)

        if val_loader is not None:
            metrics = evaluate(model, val_loader, device)
            history["val_loss"].append(metrics["loss"])
            history["val_accuracy"].append(metrics["accuracy"])
            if verbose:
                print(
                    f"epoch {epoch + 1}/{cfg.train.epochs} "
                    f"train_loss={train_loss:.4f} val_loss={metrics['loss']:.4f} "
                    f"val_acc={metrics['accuracy']:.4f}"
                )
```

```
        elif verbose:
            print(f"epoch {epoch + 1}/{cfg.train.epochs} train_loss={train_loss:.4f}")

    return history


def save_checkpoint(model: CAFN, path: str, cfg: Config = DEFAULT_CONFIG) -> None:
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    torch.save({"model_state_dict": model.state_dict(), "model_config": cfg.model},
   path)


def load_checkpoint(path: str, cfg: Config = DEFAULT_CONFIG) -> CAFN:
    checkpoint = torch.load(path, map_location="cpu", weights_only=False)
    model = CAFN(checkpoint.get("model_config", cfg.model))
    model.load_state_dict(checkpoint["model_state_dict"])
    return model


def main():
    parser = argparse.ArgumentParser(description="Train CAFN on synthetic or manifest
   data")
    parser.add_argument("--epochs", type=int, default=DEFAULT_CONFIG.train.epochs)
    parser.add_argument("--batch-size", type=int,
   default=DEFAULT_CONFIG.train.batch_size)
    parser.add_argument("--lr", type=float, default=DEFAULT_CONFIG.train.lr)
    parser.add_argument("--synthetic-samples", type=int, default=64)
    parser.add_argument("--checkpoint-name", type=str, default="cafn.pt")
    parser.add_argument("--seed", type=int, default=DEFAULT_CONFIG.train.seed)
    args = parser.parse_args()

    torch.manual_seed(args.seed)

    cfg = Config(
        train=DEFAULT_CONFIG.train.__class__(
            batch_size=args.batch_size,
            epochs=args.epochs,
            lr=args.lr,
            weight_decay=DEFAULT_CONFIG.train.weight_decay,
            aux_loss_weight=DEFAULT_CONFIG.train.aux_loss_weight,
            val_split=DEFAULT_CONFIG.train.val_split,
            seed=args.seed,
            checkpoint_dir=DEFAULT_CONFIG.train.checkpoint_dir,
        )
    )

    dataset = SyntheticNeuroDataset(num_samples=args.synthetic_samples, cfg=cfg,
   seed=args.seed)
    train_ds, val_ds = train_val_split(dataset, cfg.train.val_split, cfg.train.seed)

    model = CAFN(cfg.model)
    history = train_model(model, train_ds, val_ds, cfg=cfg, verbose=True)

    checkpoint_path = os.path.join(cfg.train.checkpoint_dir, args.checkpoint_name)
    save_checkpoint(model, checkpoint_path, cfg)
    print(f"Saved checkpoint to {checkpoint_path}")
    print(f"Final train loss: {history['train_loss'][-1]:.4f}")


if __name__ == "__main__":
    main()
```

Listing A2.11: src/explainability/grad_cam_3d.py

```
"""Grad-CAM for the 3D spatial stream of CAFN.

Backprop starts from the *fused* classification logit by default, so the
resulting activation map reflects which anatomical zones drove the
multimodal decision (not just the unimodal spatial head) -- gradients flow
from `logits_fused` back through the cross-attention fusion module into the
spatial encoder's last residual block.
"""

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F


class GradCAM3D:
    def __init__(self, model: nn.Module, target_layer: nn.Module | None = None):
        self.model = model
        self.target_layer = target_layer or model.spatial_encoder.last_conv
        self._activations = None
        self._gradients = None
        self._fwd_handle =
  self.target_layer.register_forward_hook(self._save_activation)
        self._bwd_handle =
  self.target_layer.register_full_backward_hook(self._save_gradient)

    def _save_activation(self, module, inp, out):
        self._activations = out.detach()

    def _save_gradient(self, module, grad_input, grad_output):
        self._gradients = grad_output[0].detach()

    def remove_hooks(self) -> None:
        self._fwd_handle.remove()
        self._bwd_handle.remove()

    def generate(
        self,
        mri_volume: torch.Tensor,
        vep_signal: torch.Tensor,
        target_class: int | torch.Tensor | None = None,
        logits_key: str = "logits_fused",
    ) -> tuple[torch.Tensor, torch.Tensor]:
        """Returns (cam, target_class): cam is (B, D, H, W) in [0, 1]."""
        was_training = self.model.training
        self.model.eval()
        self.model.zero_grad(set_to_none=True)

        outputs = self.model(mri_volume, vep_signal)
        logits = outputs[logits_key]

        if target_class is None:
            target_class = logits.argmax(dim=1)
        elif isinstance(target_class, int):
            target_class = torch.full(
                (logits.shape[0],), target_class, dtype=torch.long,
  device=logits.device
            )

        selected = logits.gather(1, target_class.view(-1, 1)).sum()
        selected.backward()

        if was_training:
            self.model.train()

        activations = self._activations  # (B, C, d, h, w)
        gradients = self._gradients  # (B, C, d, h, w)
        if activations is None or gradients is None:
            raise RuntimeError("Hooks did not capture activations/gradients; check
  target_layer")

        weights = gradients.mean(dim=(2, 3, 4), keepdim=True)
        cam = F.relu((weights * activations).sum(dim=1, keepdim=True))  # (B, 1, d, h,
  w)

        target_size = mri_volume.shape[2:]
        cam = F.interpolate(cam, size=target_size, mode="trilinear",
  align_corners=False)
        cam = cam.squeeze(1)  # (B, D, H, W)

        cam_min = cam.amin(dim=(1, 2, 3), keepdim=True)
        cam_max = cam.amax(dim=(1, 2, 3), keepdim=True)
        denom = (cam_max - cam_min).clamp(min=1e-8)
        cam = (cam - cam_min) / denom

        return cam.detach(), target_class.detach()
```

```
def compute_gradcam(
    model: nn.Module,
    mri_volume: torch.Tensor,
    vep_signal: torch.Tensor,
    target_class: int | torch.Tensor | None = None,
) -> tuple[torch.Tensor, torch.Tensor]:
    """One-shot convenience wrapper around GradCAM3D that cleans up its hooks."""
    cam_engine = GradCAM3D(model)
    try:
        return cam_engine.generate(mri_volume, vep_signal, target_class)
    finally:
        cam_engine.remove_hooks()


def plot_gradcam_slices(
    volume: np.ndarray,
    cam: np.ndarray,
    save_path: str,
    title: str = "Grad-CAM: Visual Cortex Activation",
) -> None:
    """Overlay a Grad-CAM heatmap on the mid axial/coronal/sagittal slices."""
    if volume.shape != cam.shape:
        raise ValueError(f"volume shape {volume.shape} != cam shape {cam.shape}")

    d, h, w = volume.shape
    slices = [
        (volume[d // 2, :, :], cam[d // 2, :, :], "Axial"),
        (volume[:, h // 2, :], cam[:, h // 2, :], "Coronal"),
        (volume[:, :, w // 2], cam[:, :, w // 2], "Sagittal"),
    ]

    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    for ax, (vol_slice, cam_slice, name) in zip(axes, slices):
        ax.imshow(vol_slice, cmap="gray")
        ax.imshow(cam_slice, cmap="jet", alpha=0.5)
        ax.set_title(name)
        ax.axis("off")
    fig.suptitle(title)
    fig.tight_layout()
    fig.savefig(save_path, dpi=150)
    plt.close(fig)
```

Listing A2.12: src/explainability/plot_attention.py

```
"""2D spatial-temporal cross-attention correlation heatmaps.

Reduces the multi-head cross-attention weights (B, H, S, T) produced by
`CrossAttentionFusion` down to a single (S, T) matrix -- spatial cortex
tokens against VEP temporal latency bins -- so clinicians can see which
brain regions the model correlated with which millisecond latency window.
"""

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch


def cross_attention_heatmap(attn_weights: torch.Tensor) -> np.ndarray:
    """Average (B, H, S, T) attention weights over batch and heads -> (S, T)."""
    if attn_weights.dim() != 4:
        raise ValueError(
            f"Expected (B, H, S, T) attention weights, got shape
   {tuple(attn_weights.shape)}"
        )
    reduced = attn_weights.mean(dim=(0, 1))
    return reduced.detach().cpu().numpy()


def plot_cross_attention(
    attn_weights: torch.Tensor,
    save_path: str,
    xlabel: str = "Temporal latency bin",
```

```
    ylabel: str = "Spatial cortex token",
    title: str = "Spatial-Temporal Cross-Attention",
) -> np.ndarray:
    """Render and save the cross-attention heatmap; returns the plotted array."""
    heatmap = cross_attention_heatmap(attn_weights)

    fig, ax = plt.subplots(figsize=(6, 8))
    im = ax.imshow(heatmap, aspect="auto", cmap="viridis")
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    fig.colorbar(im, ax=ax, label="Attention weight")
    fig.tight_layout()
    fig.savefig(save_path, dpi=150)
    plt.close(fig)

    return heatmap
```

Appendix 3
Test Inventory

A3.1 TEST FILES

Table A3.1: Test files, number of test functions and the component each exercises.

File (under `tests/`)	Tests	Component or behaviour tested
`unit/test_signal_cleaner.py`	18	VEP conditioning functions (`signal_cleaner.py`)
`unit/test_mri_transforms.py`	12	MRI preprocessing functions (`mri_transforms.py`)
`unit/test_cross_attention.py`	8	Cross-attention module (`cross_attention.py`)
`unit/test_grad_cam_3d.py`	8	3D Grad-CAM (`grad_cam_3d.py`)
`unit/test_cafn_model.py`	7	Complete model and multi-task loss (`cafn.py`)
`unit/test_dataset.py`	7	Dataset classes and synthetic generators (`dataset.py`, `synthetic.py`)
`unit/test_plot_attention.py`	5	Attention heatmap (`plot_attention.py`)
`unit/test_spatial_encoder.py`	5	Spatial encoder (`spatial_encoder.py`)
`unit/test_temporal_encoder.py`	4	Temporal encoder (`temporal_encoder.py`)
`unit/test_train_utils.py`	4	Training utilities (`train.py`)
`integration/ test_pipeline_end_to_end.py`	4	Raw files through preprocessing, model, loss, optimiser step and explainability
`integration/ test_training_loop.py`	4	Multi-epoch training, loss reduction, accuracy and checkpoint round-trip
Total	**86**	**78 unit tests and 8 integration tests**

A3.2 TEST FUNCTIONS

Listing A3.1 lists every collected test function by file, test class and name, as recorded in the repository's pytest node-identifier file.

Listing A3.1: Collected test functions.

```
unit/test_signal_cleaner.py
    TestButterworthBandpass::test_output_shape_preserved
    TestButterworthBandpass::test_attenuates_low_frequency_drift
    TestButterworthBandpass::test_preserves_inband_content
    TestButterworthBandpass::test_rejects_highcut_above_nyquist
    TestButterworthBandpass::test_rejects_lowcut_above_highcut
    TestButterworthBandpass::test_rejects_too_short_signal
    TestButterworthBandpass::test_rejects_non_1d_input
    TestButterworthBandpass::test_rejects_nan_input
    TestNotchFilter::test_attenuates_mains_hum
    TestNotchFilter::test_rejects_freq_above_nyquist
    TestZscoreNormalize::test_mean_zero_std_one
    TestZscoreNormalize::test_constant_signal_does_not_explode
    TestResampleToLength::test_upsample_length
    TestResampleToLength::test_downsample_length
    TestResampleToLength::test_identity_when_same_length
    TestCleanVepSignal::test_output_shape_dtype
    TestCleanVepSignal::test_output_is_finite_and_normalized
    TestCleanVepSignal::test_does_not_shift_p100_like_peak_latency
unit/test_mri_transforms.py
    TestLoadNifti::test_returns_ras_oriented_data
    TestLoadNifti::test_returns_voxel_zooms
    TestLoadNifti::test_rejects_non_3d_volume
```

```
    TestForegroundMaskThreshold::test_zeroes_low_intensity_voxels
    TestForegroundMaskThreshold::test_all_zero_volume_is_noop
    TestResampleIsotropic::test_anisotropic_spacing_upsamples_correct_axis
    TestResampleIsotropic::test_isotropic_input_shape_unchanged
    TestPreprocessMriVolume::test_output_shape_and_range
    TestPreprocessMriVolume::test_output_shape_default_config
    TestPreprocessMriVolume::test_constant_volume_does_not_produce_nans
    TestPreprocessMriVolume::test_rejects_non_3d_volume
    TestPreprocessMriFile::test_full_file_pipeline
unit/test_cross_attention.py
    TestMultiHeadCrossAttention::test_output_and_attention_shapes
    TestMultiHeadCrossAttention::test_attention_weights_sum_to_one
    TestMultiHeadCrossAttention::test_handles_mismatched_query_key_lengths
    TestMultiHeadCrossAttention::test_rejects_non_divisible_heads
    TestCrossAttentionFusion::test_output_shape_matches_query_tokens
    TestCrossAttentionFusion::test_output_is_finite
    TestCrossAttentionFusion::test_gradients_flow_to_all_parameters
    TestCrossAttentionFusion::test_temporal_signal_actually_influences_output
unit/test_grad_cam_3d.py
    TestGradCAM3D::test_cam_shape_matches_input_volume
    TestGradCAM3D::test_cam_values_in_unit_range
    TestGradCAM3D::test_explicit_target_class_is_respected
    TestGradCAM3D::test_hooks_are_removed_after_generate
    TestGradCAM3D::test_model_left_in_original_training_mode
    TestGradCAM3D::test_manual_engine_generate_and_remove
    TestPlotGradcamSlices::test_saves_png_file
    TestPlotGradcamSlices::test_rejects_mismatched_shapes
unit/test_cafn_model.py
    TestCAFNForward::test_output_keys_and_shapes
    TestCAFNForward::test_batch_size_one
    TestCAFNForward::test_logits_are_finite
    TestCAFNMultiTaskLoss::test_loss_is_finite_scalar
    TestCAFNMultiTaskLoss::test_loss_equals_weighted_sum_of_parts
    TestCAFNMultiTaskLoss::test_backward_reaches_every_parameter
    TestCAFNMultiTaskLoss::test_optimizer_step_changes_weights
unit/test_dataset.py
    TestSyntheticNeuroDataset::test_item_shapes_and_types
    TestSyntheticNeuroDataset::test_length
    TestSyntheticNeuroDataset::test_labels_are_balanced_across_classes
    TestSyntheticNeuroDataset::test_dataloader_batches_correctly
    TestSyntheticNeuroDataset::test_different_seeds_change_labels
    TestPairedNeuroDataset::test_reads_manifest_and_preprocesses
    TestPairedNeuroDataset::test_raises_on_empty_manifest
unit/test_plot_attention.py
    TestCrossAttentionHeatmap::test_output_shape_averages_batch_and_heads
    TestCrossAttentionHeatmap::test_values_are_nonnegative
    TestCrossAttentionHeatmap::test_rejects_wrong_dimensionality
    TestPlotCrossAttention::test_saves_png_and_returns_heatmap
    TestPlotCrossAttention::test_heatmap_matches_manual_reduction
unit/test_spatial_encoder.py
    TestSpatialEncoder3D::test_output_shape
    TestSpatialEncoder3D::test_batch_size_one
    TestSpatialEncoder3D::test_gradients_flow_to_all_parameters
    TestSpatialEncoder3D::test_last_conv_hook_target_participates_in_forward
    TestSpatialEncoder3D::test_deterministic_in_eval_mode
unit/test_temporal_encoder.py
    TestTemporalEncoder1D::test_output_shape
    TestTemporalEncoder1D::test_batch_size_one
    TestTemporalEncoder1D::test_gradients_flow_to_all_parameters
    TestTemporalEncoder1D::test_robust_to_varying_input_length
unit/test_train_utils.py
    TestTrainValSplit::test_normal_split_partitions_dataset
    TestTrainValSplit::test_single_sample_dataset_falls_back_to_all_train
    TestTrainValSplit::test_split_is_deterministic_for_fixed_seed
    TestEvaluate::test_empty_loader_returns_nan
integration/test_pipeline_end_to_end.py
    TestFullPipelineFromRawFiles::test_raw_files_to_trained_step

  TestFullPipelineFromRawFiles::test_preprocessing_output_feeds_model_without_shape_e
  rrors

  TestExplainabilityOnRealPipelineOutput::test_gradcam_and_attention_plot_from_datase
  t_sample

  TestClassConditionedSignalIsLearnable::test_glaucoma_vs_neuritis_vep_signals_are_di
  stinguishable
integration/test_training_loop.py
```

```
TestTrainModel::test_training_loss_decreases_substantially
TestTrainModel::test_model_learns_to_separate_classes
TestTrainModel::test_runs_without_validation_set
TestCheckpointRoundTrip::test_save_and_load_preserves_behavior
```

Appendix 4

Reproduction Notes

A4.1 SOFTWARE ENVIRONMENT

The repository lists its dependencies in `requirements.txt`: `torch`, `monai`, `nibabel`, `nilearn`, `mne`, `scipy`, `numpy`, `matplotlib`, `scikit-learn`, `pytest` and `pytest-cov`. Versions are not pinned; the versions installed at the time of a run should be recorded alongside its results.

A4.2 COMMANDS

Listing A4.1: Commands used with the repository.

```
# install the dependencies
pip install -r requirements.txt

# run the test suite, with coverage
pytest
pytest --cov=src

# train on synthetic data (defaults: 20 epochs, batch size 8, lr 1e-3, 64 samples)
python -m src.train --epochs 20 --batch-size 8 --lr 1e-3 \
    --synthetic-samples 64 --seed 42 --checkpoint-name cafn.pt
```

A4.3 MEASUREMENTS REPORTED IN CHAPTER 5

The signal-conditioning and synthetic-data measurements of Chapter 5 were produced by a stand-alone script, `report_experiments.py`, which imports the repository's own `clean_vep_signal`, `synthetic_vep_signal`, `synthetic_mri_volume` and `foreground_mask_threshold` functions, generates 50 seeded records per class and writes figures and a JSON file of results. A second script, `model_complexity.py`, counts parameters and multiply-accumulate operations analytically from the layer definitions. Both scripts, the figure generators and the resulting data files are supplied with this report in the folder `report_assets/`. The scripts can be re-run with `NEURO_REPO=<path to repository> python report_experiments.py <output directory>`.

A4.4 ITEMS NOT REPRODUCED HERE

- Training curves and validation metrics of the CAFN model were not produced for this report (Section 6.6); running the command in Listing A4.1 on a machine with PyTorch and MONAI installed will produce them.
- The test suite was not re-executed for this report; test counts and coverage were taken from the repository's recorded pytest and coverage data files.